Computers & Security 150 (2025) 104202

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

RanSMAP: Open dataset of Ransomware Storage and Memory Access Patterns for creating deep learning based ransomware detectors

Manabu Hirano [a,*], Ryotaro Kobayashi [b]

[a] 2-1 Eisei, Toyota, Aichi, Japan
[b] 1-24-2 Nishi-shinjuku, Shinjuku, Tokyo, Japan

ARTICLE INFO

Keywords:
Ransomware detection
Behavioral features
Deep learning
Hypervisor
Defense-in-depth

ABSTRACT

Ransomware attacks have become significant cyber threats to enterprises and public sectors. Our previous RanSAP dataset, which contained only low-level storage access patterns collected using a thin hypervisor, was used to create behavioral-based ransomware detectors; it provides an additional protection layer when the OS-level ransomware detection systems are compromised. The previous ransomware detector, which used only low-level storage access patterns, could not detect ransomware when Office applications and web browsers were executed simultaneously. This paper presents a new open dataset named RanSMAP, which stands for Ransomware Storage and Memory Access Patterns. It contains low-level storage and memory access patterns collected using a thin hypervisor. We provide an overview of the open RanSMAP dataset, including directory structure and file formats, to guide researchers in using the dataset. We then present our data preprocessing method and deep-learning-based ransomware detector. The RanSMAP datasets consist of storage and memory access patterns of six ransomware samples and six benign applications, seven Conti ransomware variants, and simultaneous execution of ransomware with benign applications collected on the machines with various CPUs, RAM generations, RAM frequencies, and RAM capacities. The experimental results show that low-level memory access patterns improved ransomware detection performance by 2.3% compared to detectors using only storage access patterns. We confirmed that ransomware detectors trained using the RanSMAP dataset can detect ransomware when Office and web browser programs are executed simultaneously. We presented the survey on state-of-the-art ransomware detection research and the availability of open behavioral-feature datasets to discuss the advantages and limitations of our RanSMAP dataset.

1. Introduction

Ransomware has become one of the most significant cyber threats to today's computer systems. Ransomware is malware that encrypts victim's files, exfiltrates confidential documents, and demands payment. Recent ransomware attacks have evolved more sophisticated than ever. Cen et al. described the ransomware attack workflow consisting of delivery, intrusion, destruction, and extortion. They presented a survey on early ransomware detection, a continuous monitoring and identification mechanism to detect ransomware attacks before the destruction phase (Cen et al., 2024). Moussaileb et al. described the ransomware workflow consisting of delivery, deployment, destruction, and dealing; they presented a survey on Windows-based ransomware detection mechanisms for each phase (Moussaileb et al., 2021). Hull et al. presented an analysis of ransomware deployment methods, including fingerprinting, propagating, Command and Control (C&C) communication, listing and encrypting files, locking OS, deleting original files, and threatening with ransom notes (Hull et al., 2019). Fig. 1 shows the detection features of defense-in-depth ransomware detection systems against multiple attack phases and attack surfaces of ransomware workflow. State-of-the-art ransomware detection mechanisms can be deployed in multiple attack phases. If a detection mechanism in a previous phase fails, another detection mechanism in the next phase will provide additional protection for the assets against ransomware attacks. This paper presents the detection mechanism in the early stage of the destruction phase (i.e., listing and encrypting victims' files), and our mechanism can be a building block for the defense-in-depth ransomware detection systems from the perspective of the attack phases and surfaces.

Oz et al. presented a survey on ransomware defense research papers with a focus on the multitude of platforms, including PCs, mobile devices, Internet of Things (IoT), and Cyber-Physical Systems (CPS) platforms (Oz et al., 2022). In their survey, 73% of the papers that

* Corresponding author.
E-mail address: hirano@toyota-ct.ac.jp (M. Hirano).

https://doi.org/10.1016/j.cose.2024.104202
Received 21 June 2024; Received in revised form 6 October 2024; Accepted 7 November 2024
Available online 16 November 2024
0167-4048/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

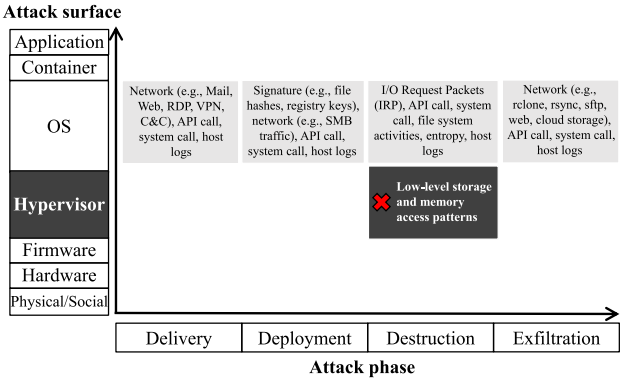


Fig. 1. Detection features of defense-in-depth ransomware detection systems in multiple attack phases and surfaces.

discuss ransomware detection on PCs have employed machine learning techniques since the signature-based techniques alone cannot detect zero-day attacks and ransomware variants; the top detection features to create machine learning models are Application Programming Interface (API) calls, file and directory operations, registry, entropy, strings, and network traffic features. Beaman et al. also surveyed recent state-of-the-art ransomware detection and prevention approaches (Beaman et al., 2021). Their survey listed the detection techniques in the destruction phase, including file analysis (e.g., file differences, file type, and file entropy) and machine learning by using file Input and Output (I/O), Opcode/Bytecode sequence, process actions, Portable Executable (PE) header, and API calls. Recent ransomware strains, however, have sophisticated anti-analysis techniques. For example, BlueSky, a variant of Conti ransomware, has API obfuscation mechanisms of indirect API calls by encoding API names using DJB hashing functions, a hash function written by Daniel J. Bernstein, to prevent API-based detection in OS layer (Palo Alto Networks, 2022). The various ransomware features enable us to build a defense-in-depth detection system shown in Fig. 1. The features to detect ransomware can be obtained from multiple layers (i.e., attack surfaces), including physical, hardware, firmware, hypervisor, operating system, container, and application layer. If an attacker gains privileged access to the operating system and evades the detection mechanisms on the OS layer, another layer has to provide an additional detection mechanism. Today's complex computer systems with multiple attack surfaces need defense-in-depth detection mechanisms. This paper presents an open dataset of low-level storage and memory access patterns obtained from the hypervisor layer to create deep-learning-based ransomware detectors, as shown at the red cross mark in Fig. 1.

Many ransomware detection systems in the literature collect static and dynamic (i.e., behavioral) features on the operating system layer (e.g., file hashes, API calls, system calls, file system activities, registry operations, and Command and Control traffic) because the ransomware strategies and actions are directly connected to the OS semantics. In contrast, the features obtained from the lower layer (e.g., hypervisor layer) have semantic gap problems. More and Tapaswi described the semantic gap problems in Virtual Machine Introspection (VMI) techniques: difficulties in deriving a complete view of the guest OS (e.g., ransomware's behaviors) from the hypervisor due to the highly dynamic nature of modern operating systems and virtualization (More and Tapaswi, 2014). This paper examines the effectiveness of the low-level behavioral features obtained from the hypervisor layer in detecting ransomware.

In addition, the ransomware detection mechanisms must be protected against anti-detection techniques. We employ low-level storage and memory features on the hypervisor layer to provide an additional protection layer against evasion attacks using, for example, privilege escalation on the OS layer. McIntosh et al. presented a survey on inadequacies in recent ransomware research and proposed a set of unified metrics to evaluate ransomware mitigation systems (McIntosh et al., 2021). The metrics include difficulty evading the detection method using anti-detection techniques, including code obfuscation and packer. Zhou et al., for example, presented the imitation attack that mimics the behavioral storage access patterns of benign applications on the OS layer to evade the machine-learning-based ransomware detectors (Zhou et al., 2023). Since the behavioral features presented in this paper include not only storage access patterns but also memory access patterns, they can provide additional protection against Zhou's imitation attack, which mimics only storage access patterns. However, detecting sophisticated evasion techniques has become more challenging than ever. We discuss the state-of-the-art evasion techniques and limitations of our RanSMAP dataset later.

In addition, a few open dynamic-feature datasets for ransomware detection have been released; Herrera-Silva and Hernández-Álvarez, for example, presented and analyzed a dynamic feature dataset for ransomware detection obtained using Cuckoo sandbox (Herrera-Silva and Hernández-Álvarez, 2023). Their paper includes the latest survey on dynamic analysis solutions for ransomware detection, and it highlighted the lack of a publicly available dataset except for our previous RanSAP dataset (Hirano et al., 2022). In addition, many dynamic analysis solutions for ransomware detection use ransomwares' behavioral features obtained on OS layer including file system activities (Kharaz et al., 2016; Scaife et al., 2016; Continella et al., 2016; Kharraz and Kirda, 2017), API calls (Kolodenker et al., 2017; Chen et al., 2017; Hampton et al., 2018; Amer and El-Sappagh, 2022; Fernando and Komninos, 2024; Singh and Tripathy, 2024), network traffic (Cabaj et al., 2018; Cusack et al., 2018; Alhawi et al., 2018), entropy analysis (Jung and Won, 2018), system logs (Roy and Chen, 2021), and their combinations (Shaukat and Ribeiro, 2018). In contrast, we released our ransomware behavioral feature dataset, RanSMAP, which was created using a thin hypervisor.

1.1. Contributions

Many ransomware detection systems have been presented considering today's complex attack phases and surfaces in ransomware workflow. Defense-in-depth detection systems enable us to increase the chance of detecting ransomware attacks with sophisticated evasion techniques. In addition, behavior-based detection techniques improve the detection rate of zero-day attacks and ransomware variants that cannot be detected only using conventional signature-based approaches. The contributions of this paper are as follows:

- We offer our new open dataset named RanSMAP, the successor to the previous dataset RanSAP (Hirano et al., 2022; Hirano, 2022). Although the previous RanSAP dataset contains only storage access patterns, the new RanSMAP dataset contains both storage and memory access patterns of six well-known ransomware samples, including the latest Conti variant, also known as BlueSky, that has an anti-detection function and six benign applications on various hardware settings. The dataset can be used to train a deep-learning-based or machine-learning-based ransomware detector. The RanSMAP dataset is available in the public repository (Hirano, 2024a,b).
- This paper offers novel analysis reports of ransomware's low-level storage and memory access patterns collected using a thin hypervisor. The machine settings used in creating the dataset are the sixth generation Intel CPU (Celeron G3920), the 12th generation Intel CPU (Core i3 12100, Core i5 12500, and Core i7 12700), six frequencies of Double Data Rate 4th generation (DDR4) RAM modules, two capacities of DDR4 RAM modules, and Double Data Rate 5th generation (DDR5) RAM modules. The dataset also includes storage and memory access patterns of ransomware

variants and those of simultaneous execution of ransomware and benign applications. The paper evaluated the performance of the ransomware detector trained using the RanSMAP dataset, analyzing the impacts of various hardware settings; this paper shows the detailed experimental results of this newly created dataset and thoroughly discusses its effectiveness and limitations.
- Providing an answer and limitations to the following research question: Can low-level storage and memory access patterns obtained using hypervisor discriminate ransomware from benign applications without solving the semantic gap problem? (i.e., can we detect ransomware using low-level storage and memory access patterns as they are?)

1.2. Relationship with previous studies

We released the RanSAP dataset (Hirano et al., 2022), which stands for Ransomware Storage Access Patterns, containing low-level storage access patterns collected using a thin hypervisor (Hirano and Kobayashi, 2019). The dataset is used to train machine-learning-based ransomware detectors. The previous ransomware detector used only low-level storage access patterns but could not detect ransomware when Office applications and web browsers were executed simultaneously. After releasing the RanSAP dataset, we presented the novel ransomware detection method using memory access patterns in addition to storage access patterns (Hirano and Kobayashi, 2022). We evaluated the performance of distinguishing only three ransomware samples and one wiper malware from three benign applications. Therefore, the detection method's effectiveness and robustness were not sufficiently evaluated. In addition, the dataset used in the previous paper (Hirano and Kobayashi, 2022) contained access patterns of only five trials on an identical machine for each sample; the number of CSV files in the previous dataset was only 210. In contrast, the number of CSV files in the new RanSMAP dataset is 11,820; in this paper, we thoroughly evaluated the effectiveness and limitations of these low-level storage and memory access patterns in various hardware settings for ransomware detection.

1.3. Structure of this paper

The rest of this paper is organized as follows. Section 2 describes the storage and memory access patterns of the RanSMAP dataset and the collection method using a thin hypervisor. Then, the directory structure and file format of the dataset are presented. Section 3 describes the ransomware samples and benign applications we used in creating the dataset. Then, the detailed hardware settings are presented. Section 4 presents our first dataset analysis. Data preprocessing method and visualization using the t-distributed Stochastic Neighbor Embedding (t-SNE) algorithm are presented. Section 5 shows the design and implementation of our baseline deep-learning-based ransomware detector consisting of a one-dimensional Convolutional Neural Network (1D-CNN) and Long Short-Term Memory (LSTM). Section 6 provides the main contribution of this paper; it presents the baseline performance of the deep-learning-based ransomware detector. Then, our experimental results of the impacts of the differences in the CPU generation, RAM frequency, RAM capacity, the number of CPU cores, and the generation of RAM modules on the ransomware detection rates are presented and thoroughly examined. To highlight the effectiveness and limitations of the RanSMAP dataset, we also evaluated the detection rates of ransomware variants using the baseline ransomware detector. We examined more realistic usage: the simultaneous execution of ransomware and benign applications. In Section 7, the comparison with the state-of-the-art ransomware detection methods and open behavioral-feature datasets, the use cases, and the limitations are presented. We summarize open issues and future work in Section 8. We conclude the summary in Section 9.

2. Overview of RanSMAP dataset

Our new dataset RanSMAP, which stands for Ransomware Storage and Memory Access Patterns, is available in the open repository (Hirano, 2024a,b). This section explains how we collected ransomware and benign applications' storage and memory access patterns using our hypervisor-based monitoring system. Then, the directory structure and file format of the RanSMAP dataset are presented.

2.1. Thin hypervisor-based monitoring system

Fig. 2 shows the schematic diagram of the hypervisor-based monitoring system, the extended version of a thin hypervisor named BitVisor (Shinagawa et al., 2009), that collects storage and memory access patterns. BitVisor is a lightweight security purpose hypervisor using Intel CPU's Virtual Machine Extensions (VMX) (Intel Corporation, 2024). BitVisor creates only a single virtual machine to monitor Input and Output (I/O) devices; I/O devices not under surveillance pass through data between the device and operating system without interactions with the hypervisor. This design of the thin hypervisor enables us to collect storage and memory access patterns as close as possible to the original condition. Our monitoring system first boots BitVisor from a write-protected USB thumb drive and then launches the Windows 10 operating system from a Serial Advanced Technology Attachment (ATA) Solid State Drive (SSD) under the surveillance of hypervisor (i.e., BitVisor). We then execute a ransomware sample or benign application on the guest OS (i.e., Windows 10). The storage access pattern monitor and the memory access pattern monitor, the extended functions of BitVisor we developed, inspect storage accesses on the Serial ATA SSD and memory accesses on RAM. The storage and memory access patterns are sent to the monitoring machine with timestamps via User Datagram Protocol (UDP) packets on a 10 Gb Ethernet data link. The received storage and memory access patterns are saved in a single Comma-Separated Values (CSV) file per ransomware or benign application execution. We replaced the Serial ATA SSD with a new one after every execution because ransomware encrypts and deletes contents on the SSD.

Storage access patterns monitor was presented in our prior work (Hirano and Kobayashi, 2019); it collects storage access patterns when the Advanced Host Controller Interface (AHCI) driver of the guest OS accesses the SSD via Direct Memory Access (DMA) protocol. BitVisor (i.e., thin hypervisor) intercepts storage accesses (i.e., data and ATA commands) between the AHCI driver of the OS and the Serial ATA SSD using a DMA shadow buffer. The DMA shadow buffer is a copy of the original guest DMA buffer, which enables us to protect the DMA buffer against attacks from the guest OS layer. The storage access patterns monitor can obtain a Logical Block Address (LBA), an operation type (i.e., *read* or *write*), the size of the accessed block in bytes, and written data. LBA is an address of the SSD that guest OS and applications access. When the operation type is *write*, the written data are used to calculate Shanon Entropy on the monitoring machine.

Memory access patterns monitor, on the other hand, was presented in our another prior work (Hirano and Kobayashi, 2022); it collects memory access patterns when the address translation from a Guest Physical Address (GPA) to a Host Physical Address (HPA) occurs in the hypervisor layer. Since BitVisor employs hardware-assisted memory virtualization named Extended Page Table (EPT) (Intel Corporation, 2024), each memory access can be intercepted when an EPT violation occurs. Once an address translation between GPA and HPA is executed, the cache is stored in the Translation Lookaside Buffer (TLB); EPT violation on the same GPA has never occurred while the cache remains in TLB. Our memory access patterns monitor in the thin hypervisor, therefore, executes a Translation Lookaside Buffer (TLB) shootdown, the technique to flush TLB buffer on all CPU cores (i.e., the cache of address translation) every 30 s. Performance degradation caused

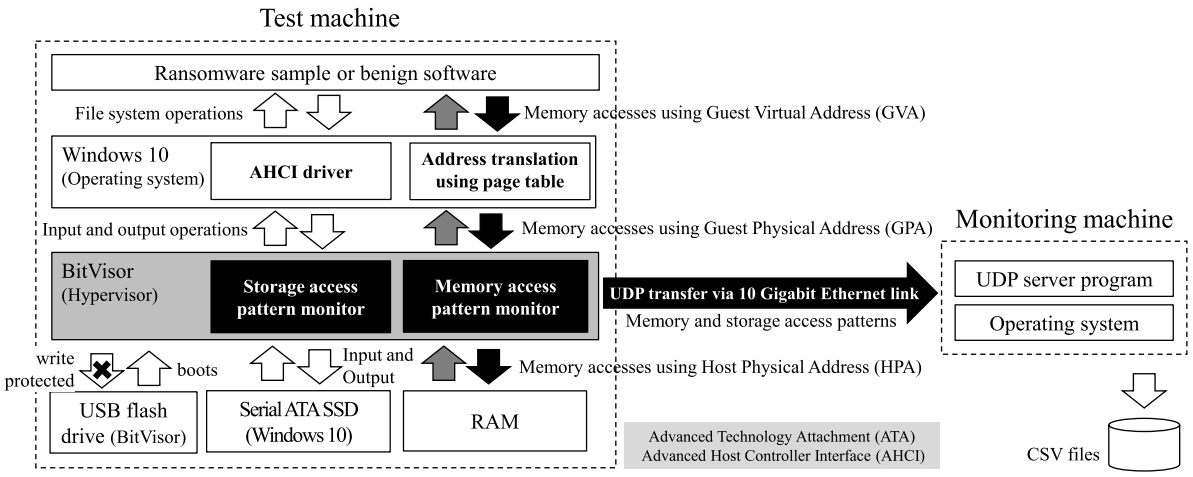


Fig. 2. Schematic diagram of thin hypervisor-based monitoring system to collect storage and memory access patterns.

by the monitoring system was examined and discussed in the prior paper (Hirano and Kobayashi, 2022).

The memory access patterns monitor obtains a guest physical address (GPA), which is identical to the Host Physical Address (HPA) in BitVisor, an operation type (i.e., *read*, *write*, *read/write*, and *execute*), an access type (i.e., 4 KiB page, 2 MiB page, 1 GiB page, and Memory Mapped Input and Output or MMIO). When the memory operation type is *write* or *read/write*, the first non-zero 4 KiB on the page is obtained; the 4 KiB data is used to calculate Shanon Entropy on the monitoring machine. The memory operation type is obtained using the Exit Qualification of a VM exit due to an EPT violation; when both read and write bits are 1, the operation type is labeled as *read/write*. When the instruction fetch bit is 1, the operation type is labeled as *execute*. For the detailed design and implementation of the memory access patterns monitor, please refer to our prior paper (Hirano and Kobayashi, 2022).

Table 1 shows the test machine's specification for collecting storage and memory access patterns. We collected access patterns of four CPUs (i.e., Celeron G3920, Core i3 12100, Core i5 12500, and Core i7 12700), those of six types of RAM modules (i.e., DDR4 1600 MHz, 1866 MHz, 2133 MHz, 2666 MHz, 3200 MHz, and DDR5 4800 MHz), two RAM capacities (i.e., 8 GiB and 16 GiB). We used two identical 8 GiB RAM modules for 16 GiB RAM tests. We employed Micron's DDR4 RAM modules (2133 MHz and 2666 MHz) and DDR5 RAM modules (4800 MHz); when we tested DDR4 RAM modules of 1866 MHz and 1600 MHz, the frequency of 2133 MHz DDR4 RAM modules were configured to 1866 MHz and 1600 MHz using UEFI firmware. A Gigabit Ethernet card was used to connect the Internet on the guest OS (i.e., Windows 10), and a 10 Gb Ethernet card was used to transfer the storage and memory access patterns to the monitoring machine. The 10 Gb Ethernet card cannot be accessed from the guest OS; BitVisor exclusively uses it and conceals it from the guest OS. The SSD of the test machine is formatted in a New Technology File System (NTFS) with GUID partition tables (GPT). Windows 10 was installed on the SSD using Unified Extensible Firmware Interface (UEFI) mode.

Fig. 3 shows the flow chart for creating the RanSMAP dataset. We created the dataset on the 15 hardware combinations; storage and memory access patterns of three minutes were collected ten times for each execution of a ransomware sample or of a benign application. We used a serial ATA drive duplicator to overwrite all the contents of the tested SSD with the initial content of the original test SSD after every execution since ransomware encrypts the original contents on the SSD.

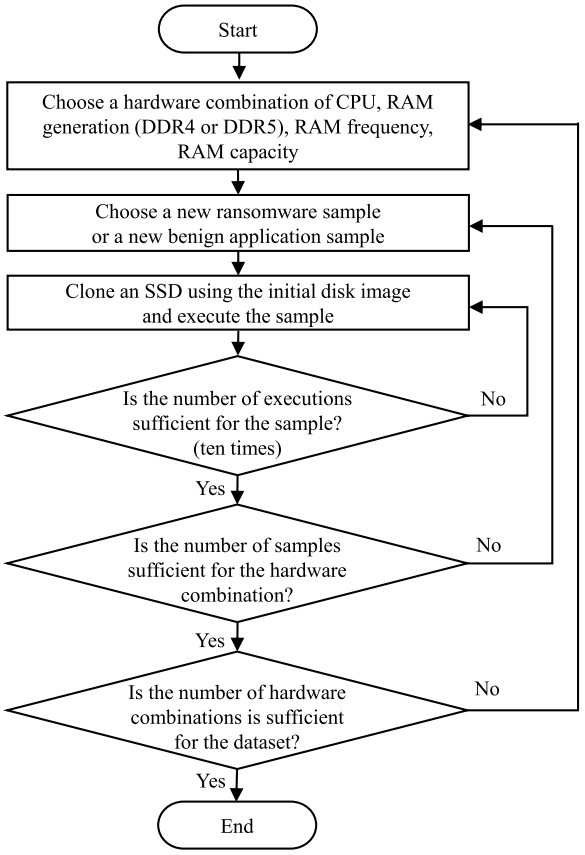


Fig. 3. Flow chart in creation of RanSMAP dataset.

2.2. Directory structure

The directory structure of the RanSMAP dataset is presented in Table 2. The storage and memory access patterns of 12 hardware combinations, including two CPUs, five RAM frequencies, and two RAM capacities, are contained in the `original` directory; we use them to train our baseline ransomware detector in Section 6.1; we analyze the impacts of CPU generations, RAM frequencies, and RAM capacities in Section 6.2. On the other hand, the `extra` directory contains three subdirectories consisting of three CPUs and two DDR generations of RAM modules; we use them to evaluate the effectiveness and limitations of the ransomware detector when the training data are collected on the machine that has different DDR generation of RAM modules and the different numbers of CPU cores in Section 6.3. Each of the 15 directories in the `original` and `extra` directories

Table 1
Specification of test machines used in collecting storage and memory access patterns.

CPU	Intel Cerelon G3920 2 cores 2 threads	Intel Core i3 12100 4 P-cores 8 threads	Intel Core i5 12500 6 P-cores 12 threads	Intel Core i7 12700 8 P-cores, 4 E-cores 20 threads	Intel Core i3 12100 4 P-cores 8 threads
	2.9 GHz	4.3 GHz	4.6 GHz	4.9 GHz	4.3 GHz
	2 MB cache	12 MB cache	18 MB cache	25 MB cache	12 MB cache
	Sixth generation	12th generation	12th generation	12th generation	12th generation
RAM	DDR4-1600 DDR4-1866 DDR4-2133	DDR4-2133 DDR4-2666 DDR4-3200	DDR4-2133	DDR4-2133	DDR5-4800
	8 GiB, 16 GiB	8 GiB, 16 GiB	16 GiB	16 GiB	16 GiB
Motherboard	ASRock H110M-HDV	ASRock B660M-HDV	ASRock B660M-HDV	ASRock B660M-HDV	ASRock B760M-HDV/M.2
Network	Intel Pro1000 (GbE), Intel X550-T1 (10 GbE)				
SSD	Crucial CT240BX (240 GB), CT250MX (250 GB), Samsung 840 (250 GB)				
OS	Windows 10 Enterprise LTSC (10.0.17763.2183)				
Hypervisor	BitVisor downloaded on 25th Jan. 2022. (8c129a1)				

Table 2
Directory structure of RanSMAP dataset.

Group		Directory name	CPU	RAM frequency	RAM capacity	Number of directories (CSV files)	Section
(1)		original/celeron-gen6/ddr4-1600-8g/	Celeron G3920	DDR4 1600 MHz	8 GiB	120 (720)	3.1, 6.1, 6.2
		original/celeron-gen6/ddr4-1600-16g/			16 GiB	120 (720)	
		original/celeron-gen6/ddr4-1866-8g/		DDR4 1866 MHz	8 GiB	120 (720)	
		original/celeron-gen6/ddr4-1866-16g/			16 GiB	120 (720)	
		original/celeron-gen6/ddr4-2133-8g/		DDR4 2133 MHz	8 GiB	120 (720)	
		original/celeron-gen6/ddr4-2133-16g/			16 GiB	120 (720)	
		original/i3-gen12/ddr4-2133-8g/	i3 12100	DDR4 2133 MHz	8 GiB	120 (720)	
		original/i3-gen12/ddr4-2133-16g/			16 GiB	120 (720)	
		original/i3-gen12/ddr4-2666-8g/		DDR4 2666 MHz	8 GiB	120 (720)	
		original/i3-gen12/ddr4-2666-16g/			16 GiB	120 (720)	
		original/i3-gen12/ddr4-3200-8g/		DDR4 3200 MHz	8 GiB	120 (720)	
		original/i3-gen12/ddr4-3200-16g/			16 GiB	120 (720)	
(2)	**(2-1)**	extra/i3-gen12/ddr5-4800-16g/	i3 12100	DDR5 4800 MHz	16 GiB	120 (720)	3.1, 6.3
	(2-2)	extra/i5-gen12/ddr4-2133-16g/	i5 12500	DDR4 2133 MHz	16 GiB	120 (720)	
	(2-3)	extra/i7-gen12/ddr4-2133-16g/	i7 12700	DDR4 2133 MHz	16 GiB	120 (720)	
(3)		variants/i3-gen12/ddr4-2133-16g/	i3 12100	DDR4 2133 MHz	16 GiB	70 (420)	3.2, 6.4
(4)		mix/i3-gen12/ddr4-2133-16g/	i3 12100	DDR4 2133 MHz	16 GiB	100 (600)	3.3, 6.5

(i.e., the group **(1)** and **(2)** in Table 2) include storage and memory access patterns of six ransomware samples and those of six benign applications; they were collected ten times for each sample, and the three-minute access patterns were saved in six CSV files per execution, as shown in Table 3. Thus, the `original` and `extra` directories contain 1,800 subdirectories and 10,800 CSV files. In addition, the `variants` directory contains storage and memory access patterns of seven Conti ransomware variants, which were first submitted to VirusTotal between 2020 and 2023; the `variants` directory contains 70 directories and 420 CSV files in total. We used them to evaluate the effectiveness and limitations of detecting ransomware variants in Section 6.4. The `mix` directory contains access patterns of ten combinations of the two ransomware samples (i.e., Conti and REvil) and the five benign applications (i.e., AESCrypt, SDelete, Zip, Firefox, and Office); the `mix` directory contains 100 subdirectories and 600 CSV files in total. They were executed simultaneously to simulate more realistic usage. We evaluated the effectiveness of our ransomware detector using these more realistic access patterns in Section 6.5. Therefore, the RanSMAP dataset contains 11,820 CSV files in total (i.e., storage and memory access patterns of 1,970 executions).

2.3. CSV file format

Table 3 shows CSV file names and formats of the RanSMAP dataset. Six CSV files are created at each execution of a ransomware or benign application sample; the six CSV files consist of two CSV files of storage access patterns and four CSV files of memory access patterns as follows. The two CSV files of storage access patterns are `ata_read.csv` and `ata_write.csv`. A line in the CSV files corresponds to a single access on an SSD; each line of the CSV files consists of the following items:

Table 3
CSV file names and formats of RanSMAP dataset.

File name	Format
`ata_read.csv`	t_s, t_{ns}, LBA, $size$, -1
`ata_write.csv`	t_s, t_{ns}, LBA, $size$, $entropy$
`mem_read.csv`	t_s, t_{ns}, GPA, 0, -1, $type$
`mem_write.csv`	t_s, t_{ns}, GPA, $size$, $entropy$, $type$
`mem_readwrite.csv`	t_s, t_{ns}, GPA, $size$, $entropy$, $type$
`mem_exec.csv`	t_s, t_{ns}, GPA, 0, -1, $type$

- UNIX Timestamp in seconds, t_s [s]
- UNIX Timestamp in nanoseconds, t_{ns} [ns]
- Logical Block Address of an accessed sector, LBA
- Size of a sector accessed by a sample, $size$ [byte]
- Normalized Shannon entropy of a written sector, $entropy$

LBA is an accessed Logical Block Address on an SSD, and $size$ is read or written byte size. We calculate $entropy$ only on a write

Table 4
Ransomware samples: SHA-256 hash values and the first submission dates at VirusTotal.

Class name	SHA-256 hash	Date
WannaCry	ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa	2017-05
Ryuk	23f8aa94ffb3c08a62735fe7fee5799880a8f322ce1d55ec49a13a3f85312db2	2018-08
REvil	0fa207940ea53e2b54a2b769d8ab033a6b2c5e08c78bf4d7dade79849960b54d	2019-04
LockBit	0e66029132a885143b87b1e49e32663a52737bbff4ab96186e9e5e829aa2915f	2020-01
Darkside	b6855793aebdd821a7f368585335cb132a043d30cb1f8dccceb5d2127ed4b9a4	2021-04
Conti	e75717be1633b5e3602827dc3b5788ff691dd325b0eddd2d0d9ddcee29de364f	2022-06

Table 5
Benign applications: version, description, and actions in creating the RanSMAP dataset.

Class name	Version	Description	Action
AESCrypt	3.10	File encryption program	Encrypting decoy files
Zip	Windows 10	File compression program	Compressing decoy files
SDelete	2.04	Secure delete program	Deleting decoy files
Firefox	98.0.2 (64 bit)	Web browser program	Accessing local web server
Office	Professional 2019	Word, Excel, and PowerPoint	Executing Word and Excel macro programs and playing PowerPoint slides
Idle	N/A	N/A	No program execution after booting Windows 10

access. Ransomware's file encryption operations increase the entropy of a written sector on a storage device; Shannon entropy, therefore, will be the crucial feature that indicates ransomware activities. $H(X)$ is a Shannon entropy of a data X, and it is calculated as follows (Shannon, 1948):

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{1}$$

where $p(x_i)$ is a probability of a byte x_i, which is an ith byte in a data X, and n is a size of the data X in byte. Since we calculate *entropy* in byte (i.e., 8 bits), Shannon entropy is finally normalized between 0 and 1 by dividing by eight. When the access was not a write operation, the *entropy* is set to −1 to distinguish from a Shannon entropy value of zero.

The four CSV files of memory access corresponding to *read*, *write*, *read/write*, and *execute* (instruction fetch) operations are `mem_read.csv`, `mem_write.csv`, `mem_readwrite .csv`, and `mem_exec.csv`. A line in the CSV files corresponds to a single access on RAM. Each line of CSV files consists of the following items:

- UNIX Timestamp in seconds, t_s [s]
- UNIX Timestamp in nanoseconds, t_{ns} [ns]
- Guest Physical Address of an accessed memory page, GPA
- Size of the obtained memory contents on the accessed page, $size$ [byte]
- Normalized Shannon entropy of a written memory contents, $entropy$
- Access type of memory page, $type$ (4 KiB page, 2 MiB page, 1 GiB page, and MMIO access)

GPA is an accessed Guest Physical Address on RAM. The thin hypervisor obtains only the first non-zero 4 KiB memory contents for entropy calculation even when the access type is 2 MiB page or 1 GiB page to avoid performance degradation. The first non-zero 4 KiB memory contents are obtained when the memory operation is *write* or *read/write*; therefore, $size$ is always equal or less than 4096 (i.e., 4 KiB) in `mem_write.csv` and `mem_readwrite.csv`, and $size$ is always zero in `mem_read.csv` and `mem_exec.csv`. Shanon entropy is calculated using the memory contents in Eq. (1). $type$ is a memory access type defined by the following integer values: 1 (4 KiB page), 2 (2 MiB page), 3 (1 GiB page), and 4 (Memory Mapped Input Output, MMIO). Since the MMIO regions on RAM are mapped to the physical peripheral devices, illegal access to them causes the abnormal halt of the computer. The thin hypervisor, therefore, does not obtain any memory contents when the memory access type is MMIO; the Shannon entropy of MMIO accesses is not calculated (i.e., both $size$ and $entropy$ are set to zero).

3. The collection

This section describes what and how ransomware samples, benign applications, and ransomware variants were executed in creating the RanSMAP dataset. The decoy files deployed on the test machine to simulate encrypting victim files are also presented. In addition, this section presents the simultaneous execution methods of a ransomware sample and benign applications to simulate more realistic usage.

3.1. Ransomware and benign applications

Table 4 presents the six ransomware samples, and Table 5 presents the six benign applications used in creating the RanSMAP dataset. Both the access patterns of the six ransomware samples and the six benign applications are included in directory groups **(1)** and **(2)** in Table 2. Table 4 shows the ransomware samples' class names, SHA-256 hash values, and the first submission date at VirusTotal (Chronicle, 2004). The six ransomware samples were obtained from ANY.RUN (ANY.RUN, 2017). We confirmed that all the ransomware samples worked properly (i.e., they encrypt victim files) in our controlled and closed environment without Internet connections to the attackers' active Command and Control (C&C) servers. We included the latest Conti ransomware variant, also known as BlueSky, which has API obfuscation mechanisms of indirect API calls to prevent API-based detection in OS layer (Palo Alto Networks, 2022). To simulate realistic ransomware attacks on business environments, we deployed decoy files on the Desktop of the Windows 10 operating system on the test machine. We copied the first 50 directories of the *Govdocs1* dataset (Garfinkel et al., 2009), which are available in the open repository (Garfinkel, 2009), to Desktop on the test machine. The names of the 50 directories in *Govdocs1* dataset are "000", "001", . . . , and "049". The *Govdocs1* dataset consists of PDF, JPEG, HTML, and Microsoft Office files collected from U.S. government websites. The number of decoy files was 49,321, totaling 26.4 GB.

Table 5 presents what and how the benign applications were executed in creating the RanSMAP dataset; we selected the benign applications that share some common behavioral features with ransomware. AESCrypt is an encryption program using the Advanced Encryption Standard (AES) symmetric encryption algorithm (Packetizer Inc., 2007). Zip is a default compression program for Windows 10 Enterprise. SDelete is a secure delete program in Microsoft's Sysinternals suite (Russinovich and Margosis, 2011). It securely deletes files, directories, partitions, and disks using the Department of defense (DOD) 5220.22-M data sanitizing standard to prevent exfiltration of confidential data by attackers. Firefox is an open-source web browser program. Microsoft Office suite includes a spreadsheet program (Excel), a word processing program (Word), and a presentation program (PowerPoint).

Table 6
Ransomware variants: SHA-256 hash values and the first submission dates at VirusTotal.

Class name	SHA-256 hash	Date
Conti #7	565ff723884f77bf7e744527b0eb736373183ce1cc6c6df0fdee4b2929f685c2	2023-08
Conti #6	1df9398f6de2d0abe0400180f3499f93bc77f6b769d68ed9f4298c488c4e48db	2022-03
Conti #5	c57d20e273337da5239f1573212adc60eacf470873e67ea135508c7c749da37b	2022-01
Conti #4	983c499dd64ba2067a00f8ed032b1b367b2cc191d0ab3573c766efdd656c2cf9	2021-02
Conti #3	3d8b6ccfcb742aeaac194c6a245ed08131a14919c4950039bf833c764e6d4f66	2021-01
Conti #2	c67ba4c6e872dbcd2b1281c33fb033f886d8472ea021cf3974a445c4b804fec2	2020-11
Conti #1	0a7e7f12d79130da067fd39ede7ff4dc3dc6665d88f5278745074d77132312bf	2020-10

Each of the AESCrypt, Zip, and SDelete programs encrypts, compresses, and deletes the decoy files of Desktop on the test machine. As described above, we extracted files from the first 50 directories of the *Govdoc1* dataset. The Firefox program accessed a web server in the same LAN the test machine deployed; the test machine accessed web pages every second using the Firefox auto-pilot plugin. The web server hosted 5752 files, and they are 1.79 GB in total; the files with ".html", ".pdf", ".jpg", and ".gif" extension were extracted from the first 50 directories (i.e., the name of the directories are "000", "001", ..., "049") of the *Govdocs1* dataset (Garfinkel et al., 2009; Garfinkel, 2009). The Office suite programs run Excel, Word, and PowerPoint simultaneously to simulate a business user's environment: The Excel program executes the macro program written in Visual Basic for Applications (VBA); it repeatedly inputs numbers on cells, creates a bar chart, and saves the Excel file. The Word program executes another macro program that repeatedly inputs texts and saves the file. The PowerPoint program plays slides repeatedly. We also collected storage and memory access patterns after booting Windows 10 but did not launch any application program (i.e., Idle class). We start the monitoring function in the hypervisor before executing benign applications except for the Office class. In the Office class, we start the monitoring function immediately after launching all the Excel, Word, and PowerPoint programs.

In creating the RanSMAP dataset, we first booted our hypervisor-based monitoring system, then booted Windows 10 under the hypervisor's surveillance. After booting Windows 10, we disabled the "real-time protection" function of Windows 10; this enables us to execute ransomware samples without being detected and removed by a Windows security scan. The directory in which ransomware samples were stored was excluded from the Windows security scan to prevent unintentional removal. Then, we start the monitoring function of BitVisor (i.e., thin hypervisor) by sending VMCALL from our utility program on the Windows operating system; the VMCALL instruction allows guest OS to make a call for a service (e.g., monitoring function) in the hypervisor. Then, a ransomware sample was executed using administrative privilege. Ransomware encrypts files on the test machine, including the decoy files on the Desktop. Each ransomware sample was executed for three minutes; the access patterns were sent from the test machine to the monitoring machine via a 10 GbE Ethernet link, as shown in Fig. 2. Then, we stopped the monitoring function in BitVisor by sending VMCALL from the utility program on the OS.

3.2. Ransomware variants

Table 6 shows SHA-256 hash values of seven Conti ransomware variants used in creating the RanSMAP dataset. The access patterns of the Conti variants are stored in the directory group **(3)** in Table 2. We selected Conti ransomware variants that were first submitted to VirusTotal (Chronicle, 2004) between 2020 and 2023; they are used to evaluate the validity of ransomware variants' behavioral features over the years. Ransomware variants were executed in the same method as described in Section 3.1.

Table 7
Simultaneous execution of a ransomware sample and benign applications.

Class name	Action
AESCrypt and Conti	Encrypting decoy files and executing Conti ransomware
SDelete and Conti	Deleting decoy files and executing Conti ransomware
Zip and Conti	Compressing decoy files and executing Conti ransomware
Firefox and Conti	Accessing local web server and executing Conti ransomware
AESCrypt and REvil	Encrypting decoy files and executing REvil ransomware
SDelete and REvil	Deleting decoy files and executing REvil ransomware
Zip and REvil	Compressing decoy files and executing REvil ransomware
Firefox and REvil	Accessing local web server and executing REvil ransomware
Office and REvil	Executing Word and Excel macro programs, playing PowerPoint slides, and executing REvil ransomware
Firefox, Office, and REvil	Accessing local web server, executing Word and Excel macro programs, playing PowerPoint slides, and executing REvil ransomware

3.3. Simultaneous execution of ransomware and benign applications

The RanSMAP dataset mainly consists of access patterns of a single sample (i.e., ransomware or benign application) execution in the controlled environment; they are stored in the directory groups **(1)**, **(2)**, and **(3)** in Table 2. The computers in actual environments, however, run multiple applications simultaneously. Therefore, we added more realistic storage and memory access patterns when ransomware samples and benign applications were executed simultaneously. Table 7 shows the ten combinations of simultaneous execution of a ransomware sample and benign applications used in creating the RanSMAP dataset; we executed each combination ten times. The storage and memory access patterns of the simultaneous executions are included in the directory group **(4)** in Table 2. Ransomware samples and benign applications were executed in the same way as described in Section 3.1. We used Conti and REvil ransomware samples shown in Table 4.

4. Analysis of RanSMAP dataset

This section provides a detailed analysis of the RanSMAP dataset through the data preprocessing method and dataset visualization.

4.1. Data preprocessing

Since the original CSV files in the RanSMAP dataset cannot be used to train deep learning models as they are, this section presents our data preprocessing method for the RanSMAP dataset; it creates fixed-length feature vectors from the original CSV files. Although the detail of the data preprocessing method of storage access patterns was presented in our prior paper (Hirano et al., 2022), and that of memory access patterns was presented in another prior paper (Hirano and

Kobayashi, 2022), this paper provides essential information to replicate the preprocessing steps accurately by other researchers, including specific algorithms and tools used, to enhance this paper's utility for the research community.

As shown in Table 3, CSV files of the RanSMAP dataset consist of lines of storage accesses and those of memory accesses. Each line of the CSV files corresponds to a single access on a storage device (i.e., SSD) and memory (i.e., RAM); therefore, each CSV file has a different number of lines (i.e., the number of accesses). Since our deep learning model needs fixed-length input vectors, our data preprocessing method transforms multiple storage and memory accesses (i.e., multiple lines in CSV files) within a time window of T_{window} s into a single 23-dimensional feature vector; the 23-dimensional feature vector consists of five-dimensional vector for storage access patterns and 18-dimensional vector for memory access patterns. The details of each feature vector are explained below.

We first describe a five-dimensional feature vector for storage access patterns. Storage access patterns were obtained using our thin hypervisor-based monitoring system as described in Section 2.1. The storage access patterns consist of the two operations: *read* and *write*; access patterns of each storage operation for a three-minute sample execution are stored in a single CSV file (i.e., `ata_read.csv` and `ata_write.csv`). The following five-dimensional feature vector is calculated from the two CSV files.

- $T_{s,r}(t)$ is an average read throughput [byte/s]
- $T_{s,w}(t)$ is an average write throughput [byte/s]
- $V_{s,r}(t)$ is a variance of Logical Block Addresses (LBAs) on which the data were read
- $V_{s,w}(t)$ is a variance of Logical Block Addresses (LBAs) on which the data were written
- $H_{s,w}(t)$ is an average Shannon entropy of written sectors

where t is the relative time elapsed after executing a ransomware or benign application sample. The preprocessing method converts t_s and t_{ns} (UNIX timestamps of the CSV files in the RanSMAP dataset) into relative time in nanoseconds since the first access on RAM or that on an SSD of each ransomware or benign application execution. $T_{s,r}(t)$ and $T_{s,w}(t)$ are the average throughput of read accesses and that of write accesses on a storage device (i.e., SSD) within the time interval $[t, t + T_{window})$. The read throughput and write throughput are used to quantify the level of file system activities of ransomware or benign applications. Variance $V_{s,r}(t)$ and $V_{s,w}(t)$ are calculated as follows:

$$V_s(t) = \frac{1}{N-1}\sum_{i=1}^{N}(LBA_i - \overline{LBA})^2 \tag{2}$$

where $\overline{LBA}$ is the mean of LBA_i, LBA_i is LBA of an ith access, and N is the number of the accesses between the time interval $[t, t + T_{window})$. Variance of LBA is used to distinguish behavioral features of ransomware's storage access patterns from benign applications' storage access patterns. Our preprocessing method calculates Shannon entropy only when the operation type is *write*. Average Shannon entropy of storage access patterns of write operations, $H_{s,w}(t)$, is calculated using *entropy* values in `ata_write.csv` file shown in Table 3 within the time interval $[t, t + T_{window})$. The average Shannon entropy of write operations, $H_{s,w}(t)$, is used to detect ransomware's encryption activities on the storage device.

Next, we describe an 18-dimensional feature vector for memory access patterns. Memory access patterns were obtained using our thin hypervisor-based monitoring system as described in Section 2.1. The memory access patterns consist of the four memory operations: *read*, *write*, *read/write*, and *execute*; access patterns of each memory operation for three-minutes sample execution are stored in four CSV files (i.e., `mem_read.csv`, `mem_write.csv`, `mem_readwrite.csv`, and `mem_exec.csv`). The following five groups of feature vectors are calculated using the four CSV files; since our preprocessing method calculates Shannon entropy only when the operation type is *write* or *read/write*, we finally obtain 18-dimensional vectors for memory access patterns.

- Average Shannon entropy of a written page: $H_{m,w}(t)$ and $H_{m,rw}(t)$
- The number of accessed 4 KiB pages: $C_{4KB,r}(t)$, $C_{4KB,w}(t)$, $C_{4KB,rw}(t)$, and $C_{4KB,x}(t)$
- The number of accessed 2 MiB pages: $C_{2MB,r}(t)$, $C_{2MB,w}(t)$, $C_{2MB,rw}(t)$, and $C_{2MB,x}(t)$
- The number of accessed MMIO pages: $C_{MMIO,r}(t)$, $C_{MMIO,w}(t)$, $C_{MMIO,rw}(t)$, and $C_{MMIO,x}(t)$
- Variance of accessed Guest Physcal Addresses (GPAs), $V_{m,r}(t)$, $V_{m,w}(t)$, $V_{m,rw}(t)$, and $V_{m,x}(t)$

where t is the relative time elapsed after executing a ransomware or benign application sample. The preprocessing method converts t_s and t_{ns} (UNIX timestamps of the CSV files in the RanSMAP dataset) into relative time in nanoseconds since the first access on RAM or that on an SSD of each ransomware or benign application execution. Average Shannon entropy of memory access patterns of *write* and *read/write* operations, $H_{m,w}(t)$ and $H_{m,rw}(t)$, are calculated using *entropy* values of `mem_write.csv` and `mem_readwrite.csv` files shown in Table 3 within the time interval $[t, t+T_{window})$. The average Shannon entropy of write memory operations is used to identify ransomware's encryption activities on RAM. Also, we counted the number of accesses for each access type (i.e., 4 KiB, 2 MiB, and MMIO) and each operation type (i.e., *read*, *write*, *read/write*, and *execute*). Please note that the current monitoring system does not count the number of accesses when the access type is 1 GiB since we have not observed 1 GiB page access on Windows 10. For example, $C_{2MB,r}(t)$ is the number of accesses when the access type is 2 MiB page, and the memory operation type is *read*. On the other hand, $C_{MMIO}(t)$ is the number of accesses when the access type is Memory Mapped Input Output (MMIO); this counter value indicates a level of input and output operations of an external device such as SSD. In contrast, $C_{4KB}(t)$ and $C_{2MB}(t)$ indicate the amount and frequency of memory accesses on physical RAM. A variance of the accessed Guest Physical Address (GPA), $V_m(t)$, is calculated in Eq. (2), but using GPA instead of LBA. A variance of GPA, $V_m(t)$, is used to distinguish memory access patterns of different programs.

Our preprocessing method finally creates a single 23-dimensional feature vector, $x(t)$, consisting of the five-dimensional features of storage access patterns and the 18-dimensional features of memory access patterns between the time interval $[t, t + T_{window})$. A series of the 23-dimensional features is represented as $\chi = \{x(0), x(\epsilon), x(2\epsilon), x(3\epsilon), ..., x(T_d - T_{window})\}$. We also define the two subsets of $\chi = \{\chi_s, \chi_m\}$: a series of the five-dimensional storage feature vectors χ_s and that of the 18-dimensional memory feature vectors χ_m.

T_d is the duration of access patterns the deep learning model uses in detecting ransomware. For example, we can use the access patterns of the first 30 s after executing a ransomware sample to train a deep-learning-based ransomware detector (i.e., T_d = 30 s). The ransomware detector can detect ransomware in 30 s after execution of ransomware; T_d is, therefore, referred to as the detection time of ransomware or duration of access patterns used in creating a deep learning model. Although the ransomware detector trained using access patterns of smaller T_d can detect ransomware quickly, it decreases the detection performance. We evaluated the relationship between T_d and the ransomware detection rate in Section 6. $x(0)$ is a single 23-dimensional feature vector calculated in $[0, T_{window})$ (i.e., a feature vector of access patterns immediately after executing a ransomware sample or benign application). We tested four T_{window} values: 10 s, 1 s, 0.1 s, and 0.01 s. While we use $\epsilon = T_{window}$ when T_{window} is 1 s, 0.1 s, and 0.01 s, we use ϵ = 1 s when T_{window} is 10 s since we cannot create sufficient number of feature vectors (i.e., we can obtain only three 23-dimensional feature vectors in T_d = 30 s when we use $\epsilon = T_{window}$). We compared the effects of T_{window} in Section 6.

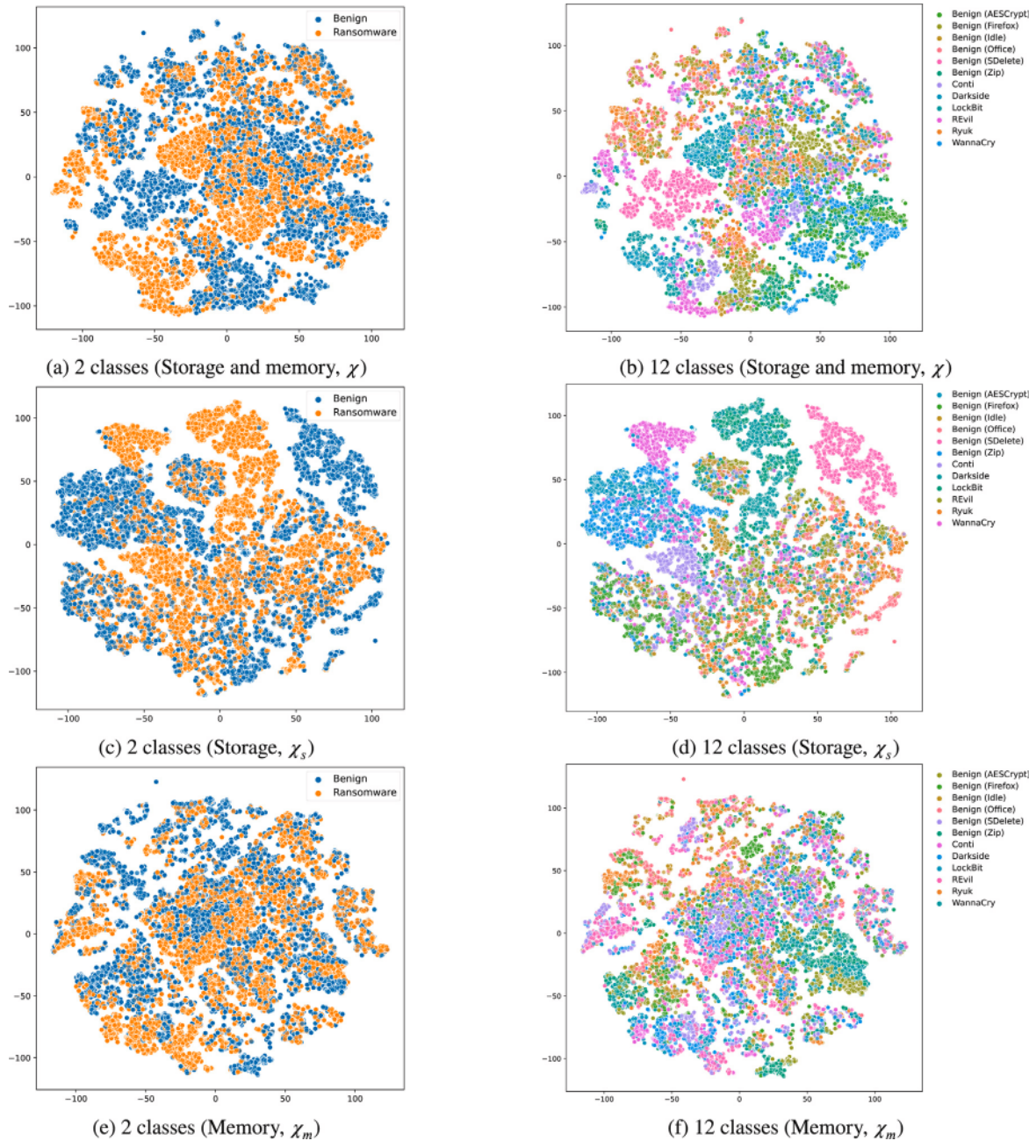


Fig. 4. Visualization by t-SNE (T_{window} = 10 s and T_d = 30 s).

Algorithms and tools. Finally, we show our data preprocessing algorithms to create storage feature χ_s and memory feature χ_m in Figs. D.1 and D.2, respectively. We implemented the two data preprocessing algorithms using Python 3.11.5 and NumPy 1.24.3 on Ubuntu 22.04.3 LTS. In Fig. D.1, all lines of two files (i.e., `ata_read.csv` and `ata_write.csv`) are read (lines 3 and 4), lines in a time interval $[t, t + T_{window})$ are extracted (lines 6 and 7), five-dimensional feature vector $x_s(t)$ is calculated (lines 8–13), and $x_s(t)$ is appended to storage feature χ_s (line 14). The feature calculation is repeated within the specified duration of access patterns, T_d (lines 5–15). In Fig. D.2, all lines of four files (i.e., `mem_read.csv`, `mem_write.csv`, `mem_readwrite.csv`, and `mem_exec.csv`) are read (lines 3–6), lines in a time interval $[t, t+T_{window})$ are extracted (lines 8–11), 18-dimensional feature vector $x_m(t)$ is calculated (lines 12–18), and $x_m(t)$ is appended to memory feature χ_m (line 19). The feature calculation is repeated within the specified duration of access patterns, T_d (lines 7–20).

4.2. Visualization of RanSMAP dataset

Dimensionality reduction algorithms convert high dimensional features into two or three-dimensional features, and the algorithms are often used to visualize the characteristics of the high-dimensional datasets. The t-SNE (van der Maaten and Hinton, 2008) algorithm is a dimensionality reduction method that can preserve the significant structure of high-dimensional original feature vectors. We used the t-SNE algorithm to reduce the dimensionality of the 23-dimensional feature vectors into two-dimensional feature vectors for visualization. We first created feature vectors using the CSV files of the directory group **(1)** shown in Table 2. We used the parameter of T_{window} = 10 s and T_d = 30 s in our data preprocessing method. Fig. 4 shows the results of applying t-SNE to a series of the feature vectors, χ, χ_s, and χ_m. χ is a series of the 23-dimensional feature vectors, χ_s is that of the five-dimensional storage feature vectors, and χ_m is that of the

18-dimensional memory feature vectors described in Section 4.1. We created the scatter plots of the two classes (i.e., ransomware or benign application) and the 12 classes (i.e., the class names of ransomware sample and benign application). In the scatter plots of the two classes, the ransomware class contains the six ransomware samples shown in Table 4, and the benign class contains the six benign applications shown in Table 5.

The scatter plots (a), (c), and (e) in Fig. 4 show visualization of feature vectors in two classes. Although we created feature vectors using 12 combinations of the hardware settings (i.e., two CPUs, five memory frequencies, and two memory capacities), the two-dimensional scatter plot of the two classes (a), (c), and (e) show visible clusters of the RanSMAP dataset; it indicates that we can use the feature vectors to discriminate between behavioral patterns of ransomware samples from those of benign applications. The scatter plot (c) of five-dimensional storage feature vectors χ_s shows clearer clusters than the scatter plot (e) of 18-dimensional memory feature vectors χ_m; the scatter plot (a) that uses storage and memory feature vectors χ also shows some visible clusters. The scatter plots (b), (d), and (f) in Fig. 4 show visualization of feature vectors in 12 classes. The scatter plot (d) that uses storage feature vectors χ_s shows clearer clusters than that of (f) that uses memory feature vectors χ_m; it indicates that storage access patterns have better classification performance than memory access patterns. The scatter plot (b) that uses storage and memory feature vectors χ also shows some clear clusters.

As shown in Fig. 4, we confirmed that storage feature vectors χ_s show clearer clusters than memory feature vectors χ_m. So why do we need memory access patterns? That is because some attacks cannot be detected using only storage features. The imitation attacks (Zhou et al., 2023), for example, mimic benign applications' Input and Output (I/O) behaviors to evade ransomware detection systems that use only I/O behaviors. Furthermore, emerging fileless malware (Barr-Smith et al., 2021) and living-off-the-land malware (Kumar et al., 2020) do not leave significant I/O activities on storage devices; such state-of-the-art attacks makes the detection using only I/O behaviors challenging. We, therefore, included memory access patterns in addition to storage access patterns in the new RanSMAP dataset. We can see that the scatter plots that use both storage and memory access patterns, (a) and (b), are slightly improved compared to those using only storage memory access patterns and those using only memory access patterns. We thoroughly examine the effects of using storage and memory access patterns in Section 6.

5. Deep-learning-based ransomware detector

This section presents our ransomware detector using a deep-learning model. Fig. 5 shows an overview of the deep learning model consisting of a one-dimensional Convolutional Neural Network (1D-CNN) layer and two Long Short-Term Memory (LSTM) layers. Keras 2.12.0 was used to build the deep learning model. After testing combinations in the number of LSTM layers, the number of LSTM units, the number of CNN layers, and the kernel size of 1D-CNN, we decided on the following deep-learning model and hyperparameters. The 23-dimensional features χ (i.e., storage and memory access patterns) are first fed into the 1D-CNN layer to convolve them over a temporal dimension by each feature. When we use only the storage access features χ_s, or memory features χ_m, five-dimensional or 18-dimensional features are fed into the five or 18 1D-CNNs instead of 23 1D-CNNs. Thus, the first CNN layer has the number of 1D-CNNs equal to the number of the dimensions of feature vectors. We employed a kernel size 10 and the linear activation function in Keras's *Conv1D* layer for each 1D-CNN. Five, 18, or 23 Max-pooling operations are then applied to the outputs of the 1D-CNNs; we used a pool size 2 of Keras's *MaxPooling1D* layer for each Max-pooling operation. The outputs of the *MaxPooling1D* operations are then fed into the LSTM layers. We used Keras's *LSTM* layer with 256 units; the first LSTM layer outputs complete sequence,

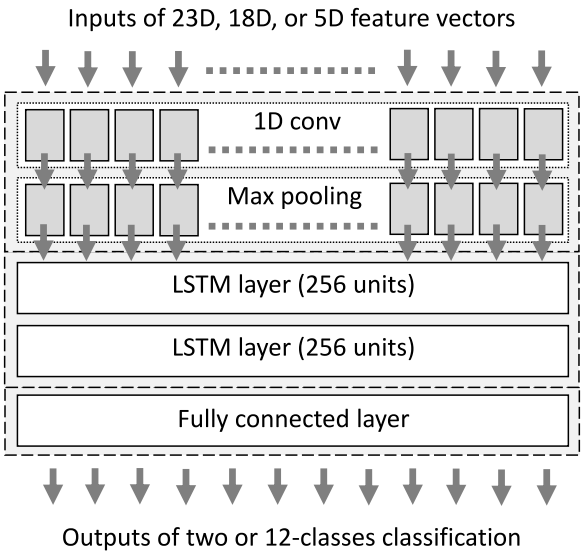


Fig. 5. Overview of our deep-learning-based ransomware detector consisting of a one-dimensional Convolutional Neural Network (1D-CNN) layer, a Max pooling layer, and two Long Short-Term Memory (LSTM) layers.

and then the next LSTM layer passes the outputs to the next fully connected layer (i.e., Keras's *Dense* layer). The number of the fully connected layer's outputs is configured to two for binary classification (i.e., ransomware or benign applications) and 12 for 12-classes classification (i.e., ransomware class names and benign application class names). The softmax activation function was used in the last fully connected layer. Categorical cross entropy was used for the loss function.

6. Evaluation of deep learning based ransomware detector using RanSMAP dataset

This section shows the performance of the deep-learning-based ransomware detector presented in Section 5 to demonstrate the effectiveness and limitations of the RanSMAP dataset. Fig. 6 shows the relationship between the subsections of Section 6 and RanSMAP dataset groups. In Section 6.1, we first created and evaluated our *baseline* ransomware detector trained using the entire dataset group **(1)**, including storage and memory access patterns collected on the machines using two CPUs, five RAM frequencies, and two RAM capacities. The baseline ransomware detector is later used in Sections 6.3, 6.4 and 6.5. In Section 6.2, the impacts of the hardware differences of CPU generations, RAM frequency, and RAM capacity on ransomware detection rates using only specific parts of the dataset group **(1)** (i.e., dataset group **(1-1)**, **(1-2)**, **(1-1-1)**-**(1-2-3)**, **(1-a)**, **(1-b)**), which are shown in Table 8, are examined. In Section 6.3, we evaluated the impacts of the DDR RAM module generation and the number of CPU cores using the baseline ransomware detector. In Section 6.4, we evaluated the detection performance of ransomware variants using the baseline ransomware detector. In Section 6.5, we further examined the detection performance of ransomware in more realistic usage, simultaneous execution of ransomware, and benign applications using the baseline ransomware detector.

We employed the F score to measure the performance of ransomware detectors. F score is a harmonic mean of precision and recall; it is calculated as follows:

$$Fscore = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \tag{3}$$

Precision and recall were calculated as follows:

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

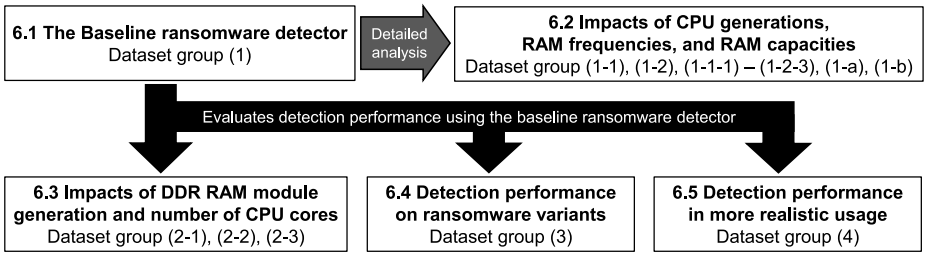
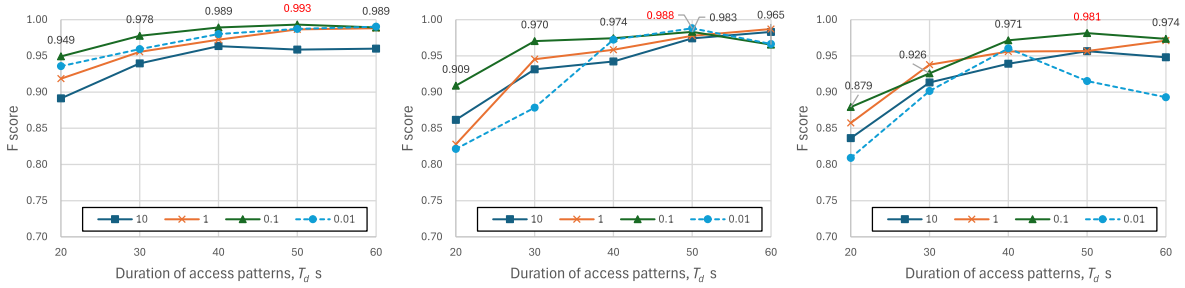


Fig. 6. The relationship between subsections of Section 6 and RanSMAP dataset groups. The dataset groups are shown in Tables 2 and 8.



(a) Storage and memory, χ (2 classes) (b) Storage, χ_s (2 classes) (c) Memory, χ_m (2 classes)

Fig. 7. Comparison of F scores in 2 classes by T_d = 20, 30, 40, 50, and 60 s and T_{window} = 10, 1, 0.1, and 0.01 s. The data labels are displayed only for T_{window} = 0.1 s, except for the highest F score in red. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

where TP (True Positive) is the number of correctly classified features in the correct class. FP (False Positive) is the number of incorrectly classified features in the correct class. FN (False Negative) is the number of incorrectly classified features that are not in the correct class. We employed *micro* option of Scikit-learn's *metrics.f1_score* class to calculate F scores, which globally counts the total TPs, FNs, and FPs over classes.

6.1. The baseline ransomware detector

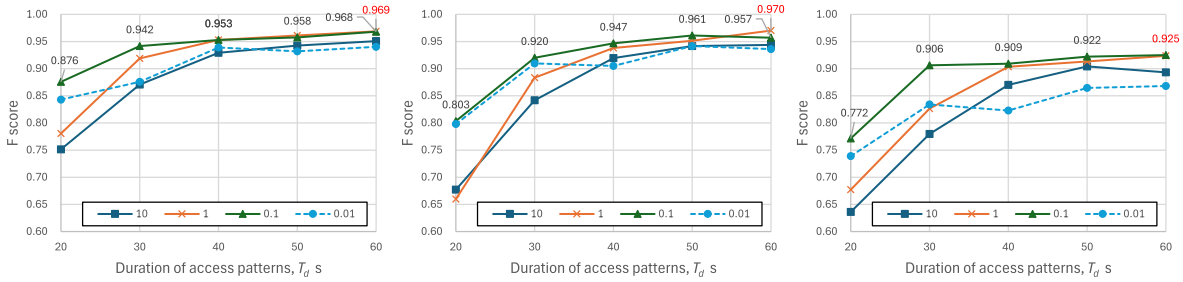
We first created the baseline ransomware detector for analysis in the subsequent sections. The baseline ransomware detector was trained using group **(1)** of the RanSMAP dataset shown in Table 2. The group **(1)** includes the access patterns collected on the machine with two CPUs (the sixth generation Intel CPU and the 12th generation CPU), five RAM frequencies (1600 MHz, 1866 MHz, 2133 MHz, 2666 MHz, and 3200 MHz), and two RAM capacities (8 GiB and 16 GiB).

Fig. 7 shows ransomware and benign applications' F scores in binary classification (i.e., two classes). We created three types of detectors using storage and memory features χ, storage features χ_s, and memory features χ_m. Each detector is trained using five combinations of the duration of access patterns (i.e., T_d = 20, 30, 40, 50, and 60 s) and four combinations of the time of window (i.e., T_{window} = 10, 1, 0.1, and 0.01 s). Each detector was trained five times to compensate for variations in the deep learning models; we thus created 100 detectors (i.e., deep learning models shown in Section 5) for each of three detector types (i.e., storage and memory, storage, and memory). Each detector was trained using 80% of the group **(1)** in Table 2 and tested using the remaining 20% of the group **(1)** to calculate F scores.

In Fig. 7, the highest F scores are 0.993 at T_d = 50 s and T_{window} = 0.1 s when both storage and memory features χ were used, 0.988 at T_d = 50 s and T_{window} = 0.01 s when only storage features χ_s were used, and 0.981 at T_d = 50 s and T_{window} = 0.1 s when only memory features χ_m were used. We calculated mean F scores using all combinations of T_d and T_{window} for each feature set of χ, χ_s, and χ_m. The mean F score was 0.964 when both storage and memory features χ were used, followed by the mean F score of 0.941 when only storage features χ_s were used and the mean F score of 0.924 when only memory features χ_m were used. We confirmed that adding memory features χ_m to storage features χ_s improved 2.3% on the mean F score in binary classification. The results indicated that the storage features χ_s and memory features χ_m complement each other; we confirmed that both features contribute to improving the F scores of our baseline ransomware detector.

Fig. 8 shows the F scores in the 12-classes classification; the 12 classes consist of the six ransomware samples shown in Table 4 and the six benign applications shown in Table 5. In Fig. 8, the highest F scores are 0.969 at T_d = 60 s and T_{window} = 1 s when both storage and memory features χ were used, 0.970 at T_d = 60 s and T_{window} = 1 s when only storage features χ_s were used, and 0.925 at T_d = 60 s and T_{window} = 0.1 s when only memory features χ_m were used. The mean F score was 0.913 when both storage and memory features χ were used, followed by the F score of 0.890 when only storage features χ_s were used, and the F score of 0.844 when only memory features χ_m were used. The results in 12-classes classification also indicated that the storage features χ_s and memory features χ_m complement each other; we confirmed that adding memory features χ_m to storage features χ_s also improved 2.3% on the mean F score in 12-classes classification as in binary classification. We thus confirmed that both storage and memory features contribute to improving the F scores of our ransomware detector.

Finally, we determined the optimal parameters of T_d and T_{window} of our *baseline* ransomware detector for the following experiments. The duration of access patterns T_d means the time required to detect ransomware. Therefore, as the value of T_d increases, it takes more time to detect ransomware; that means ransomware can encrypt more files. In this paper, we chose T_d = 30 s, balancing F score and detection time. Since modern ransomware strains employ high-speed encryption using multi-threading, we aim for detection in 30 s or less. In Fig. 7(a), the best F score in 2 classes was 0.993 when T_d = 50 s and T_{window} = 0.1 s was used. The F score of T_d = 30 s and T_{window} = 0.1 s was 0.978, only a 1.5% decrease compared to the best F score. However, the F score of T_d = 20 s and T_{window} = 0.1 s was 0.949, a 4.4% decrease compared to the best F score. We therefore decided to choose T_d = 30 s. In addition, we chose T_{window} = 0.1 s since the best F score was always obtained



(a) Storage and memory, χ (12 classes) (b) Storage, χ_s (12 classes) (c) Memory, χ_m (12 classes)

Fig. 8. Comparison of F scores in 12 classes by T_d = 20, 30, 40, 50, and 60 s and T_{window} = 10, 1, 0.1, and 0.01 s. The data labels are displayed only for T_{window} = 0.1 s except for the highest F score. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 8
Dataset subgroups used to evaluate impacts of different hardware settings on F scores.

	Directory name	CPU		RAM frequency		RAM capacity	
(1)	original/celeron-gen6/ddr4-1600-8g/ original/celeron-gen6/ddr4-1600-16g/	Celeron G3920	**(1-1)**	DDR4 1600 MHz	**(1-1-1)**	8 GiB 16 GiB	**(1-a)** **(1-b)**
	original/celeron-gen6/ddr4-1866-8g/ original/celeron-gen6/ddr4-1866-16g/			DDR4 1866 MHz	**(1-1-2)**	8 GiB 16 GiB	**(1-a)** **(1-b)**
	original/celeron-gen6/ddr4-2133-8g/ original/celeron-gen6/ddr4-2133-16g/			DDR4 2133 MHz	**(1-1-3)**	8 GiB 16 GiB	**(1-a)** **(1-b)**
	original/i3-gen12/ddr4-2133-8g/ original/i3-gen12/ddr4-2133-16g/	Core i3 12100	**(1-2)**	DDR4 2133 MHz	**(1-2-1)**	8 GiB 16 GiB	**(1-a)** **(1-b)**
	original/i3-gen12/ddr4-2666-8g/ original/i3-gen12/ddr4-2666-16g/			DDR4 2666 MHz	**(1-2-2)**	8 GiB 16 GiB	**(1-a)** **(1-b)**
	original/i3-gen12/ddr4-3200-8g/ original/i3-gen12/ddr4-3200-16g/			DDR4 3200 MHz	**(1-2-3)**	8 GiB 16 GiB	**(1-a)** **(1-b)**

when T_{window} = 0.1 s was used in Fig. 7(a). In the rest of this section, we used the parameters of T_d = 30 s and T_{window} = 0.1 s. We included the confusion matrices of the baseline ransomware detector created using the above parameters in Appendix B to inspect the detection performance of the baseline ransomware detector in detail.

6.2. Impacts of CPU generations, RAM frequencies, and RAM capacities

This section examines the impacts of the differences in hardware settings (i.e., CPU generations, RAM frequencies, and RAM capacities) on the ransomware detector's F scores. We used both storage and memory features χ. Each ransomware detector was trained five times to compensate for variations in the deep learning models. Table 8 shows subgroups of the group **(1)** in the RanSMAP dataset. For example, we trained the ransomware detector using the dataset of subgroup **(1-1)**, the access patterns of the sixth generation Intel CPU (i.e., Celeron G3920), then tested the ransomware detector using the dataset of subgroup **(1-2)**, the access patterns of the 12th generation Intel CPU (i.e., Core i3 12100), to evaluate how the ransomware detector trained using the access patterns of the older CPU (i.e., D_{train} = {**(1-1)**}) can detect ransomware using those of the newer CPU (i.e., D_{test} = {**(1-2)**}), and vice versa.

First, we examined the effects of RAM frequency. Fig. 9 shows F scores of the ransomware detectors trained using both storage and memory features χ excluding a RAM frequency subgroup from **(1-1-1)** to **(1-2-3)**. We trained the baseline ransomware detectors, shown in the most left two bars, for two classes and 12 classes using randomly selected 85% of the group **(1)** in Table 8, and then tested the detector using the remaining 15% of the group **(1)**. The F scores of the two baseline ransomware detectors for binary and 12-classes classification at the most left in Fig. 9 were used to evaluate the effects of the hardware differences.

The lowest F scores were obtained both for two classes and 12 classes when we trained the ransomware detector using the dataset excluding **(1-2-1)**, which are access patterns on DDR4 2133 MHz RAM modules with Core i3 12100 CPU (i.e., D_{test} = {**(1-2-1)**} and D_{train} = {**(1-1-1)**, **(1-1-2)**, **(1-1-3)**, **(1-2-2)**, **(1-2-3)**}); the lowest F scores were 0.948 for two classes (i.e., 3.4% decrease compared to the baseline ransomware detector) and 0.878 for 12 classes (i.e., 6.2% decrease compared to the baseline ransomware detector). In contrast, we obtained the best F scores both for two classes (0.1% increase compared to the baseline ransomware detector) and 12 classes (0.8% increase compared to the baseline ransomware detector) when we trained the detector using the dataset excluding **(1-1-3)**, which are the access patterns on DDR4 2133 MHz modules with Celeron G3920 CPU. We could not determine the reason why the F scores were slightly improved when we excluded **(1-1-3)** from the training dataset; the access patterns of the same RAM frequency on the newer CPU **(1-2-1)** might have helped to detect those of the same RAM frequency on the older CPU **(1-1-3)**. We, however, did not observe a significant decrease in F scores (i.e., the decrease rate was less than or equal to 3.4% in binary classification) in this experiment. Thus, we confirmed that the difference in RAM frequency has a limited impact on F scores.

Next, we examined the impact of different CPU generations on F scores. Fig. 10 shows F scores of the ransomware detectors trained using storage and memory features χ excluding a subgroup of different CPU generation. Excluding the subgroup **(1-1)** of the older sixth-generation Intel CPU (i.e., Celeron G3920) from the training dataset caused a 2.9% decrease on F score compared to the baseline ransomware detector in binary classification (i.e., D_{test} = {**(1-1)**} and D_{train} = {**(1-2)**}). In contrast, excluding the subgroup **(1-2)** of the newer 12th generation Intel CPU (i.e., Core i3 12100) caused a more significant decrease of 12.1% on F score compared to the baseline ransomware detector in binary classification (i.e., D_{test} = {**(1-2)**} and D_{train} = {**(1-1)**}). In 12 classes, excluding the subgroup **(1-1)** from the training dataset caused a 16.3% decrease (i.e., D_{test} = {**(1-1)**} and D_{train} = {**(1-2)**}) while excluding the subgroup **(1-2)** from the training dataset caused a more

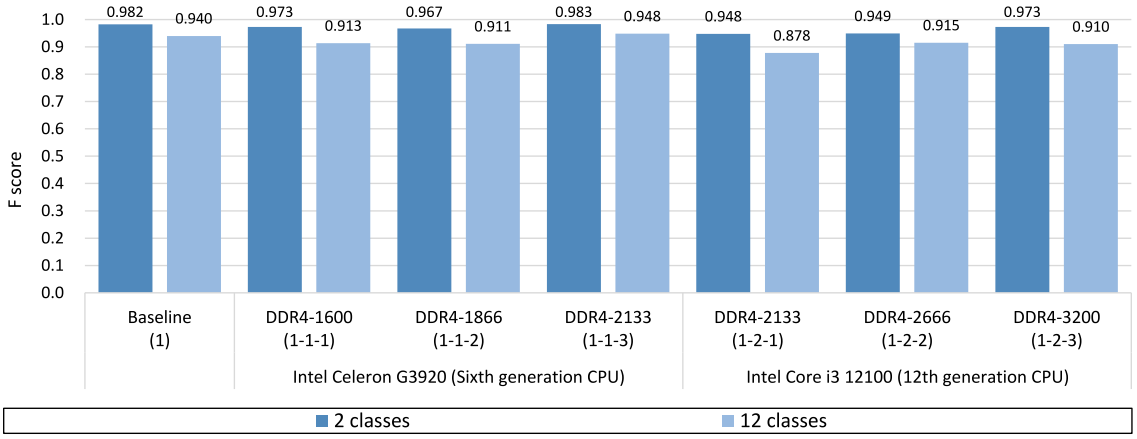


Fig. 9. F scores of the ransomware detectors trained using storage and memory features χ excluding a RAM frequency subgroup. The excluded subgroups are shown in horizontal axis titles, except for the baseline ransomware detector.

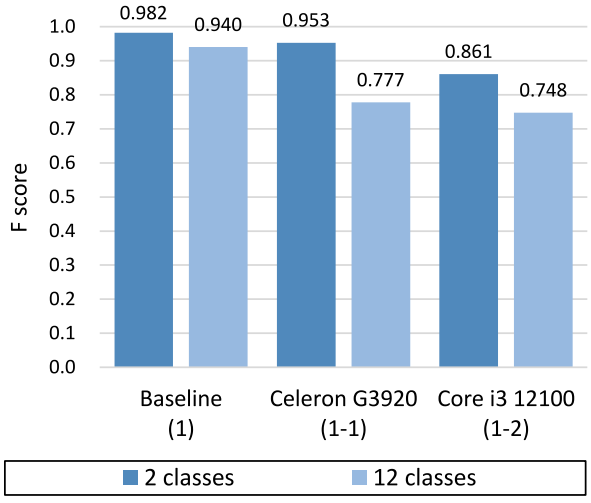


Fig. 10. F scores of the ransomware detectors trained using storage and memory features χ excluding a CPU subgroup. The excluded subgroups are shown in horizontal axis titles, except for the baseline ransomware detector.

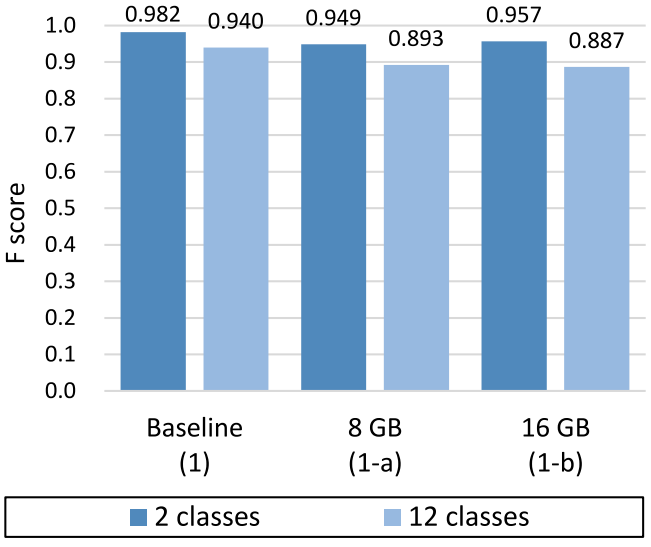


Fig. 11. F scores of the ransomware detectors trained using storage and memory features χ excluding a RAM capacity subgroup. The excluded subgroups are shown in horizontal axis titles, except for the baseline ransomware detector.

significant decrease of 19.2% on F score compared to the baseline ransomware detector (i.e., D_{test} = {**(1-2)**} and D_{train} = {**(1-1)**}).

Although the experiments were conducted under the limited conditions of only two CPUs, we confirmed that the ransomware detector in binary classification (i.e., two classes) trained using access patterns of the newer CPU (the 12th generation Intel CPU) was able to precisely detect the access patterns of the older CPU (the sixth generation Intel CPU); the detector trained using the access patterns on the older CPU had limited ability to detect ransomware executed on the newer CPU. Thus, we confirmed that the difference between the CPU generations impacted F scores over RAM frequencies. We included the confusion matrices of the baseline ransomware detector testing the two CPU generations in Appendix C to inspect the impacts in detail.

We examined the impact of the different RAM capacities on F scores. Fig. 11 shows the ransomware detectors' F scores trained using storage and memory features χ excluding a RAM capacity subgroup. In binary classification, excluding the subgroup **(1-a)** of 8 GiB RAM capacity from the training dataset caused a 3.3% decrease compared to the baseline ransomware detector (i.e., D_{test} = {**(1-a)**} and D_{train} = {**(1-b)**}), while excluding **(1-b)** of 16 GiB RAM capacity from the training dataset caused a 2.5% decrease compared to the baseline ransomware detector (i.e., D_{test} = {**(1-b)**} and D_{train} = {**(1-a)**}). In 12 classes, excluding **(1-a)** caused a 4.7% decrease (i.e., D_{test} = {**(1-a)**} and D_{train} = {**(1-b)**}), while excluding **(1-b)** caused a 5.3% decrease (i.e., D_{test} = {**(1-b)**} and D_{train} = {**(1-a)**}). In binary classification, we did not observe a significant decrease in F scores of more than 3.3% when we trained the ransomware detector using the access patterns excluding a subgroup of RAM capacity. Thus, we confirmed that the difference in RAM capacity has a limited impact on F scores.

6.3. Impacts of DDR RAM module generation and number of CPU cores

This section examines the impacts of the generation of DDR RAM modules (i.e., Double Data Rate RAM modules of the 4th generation and those of the 5th generation) and the different number of CPU cores in the same generation of Intel CPUs (i.e., the 12th generation Intel Core i3 12100, Core i5 12500, and Core i7 12700). Each detector was trained using storage and memory features χ, storage features χ_s, and memory features χ_m of the group **(1)** in Table 2. Then, we tested the detector to predict a class using the features of a subgroup **(2-1)**, **(2-2)**, and **(2-3)** in Table 2 (i.e., D_{test} = {**(2-1)**} and D_{train} = {**(1)**}, D_{test} = {**(2-2)**} and D_{train} = {**(1)**}, and D_{test} = {**(2-3)**} and

D_{train} = {**(1)**}), which are the access patterns of 5th generation of Double Data Rate (DDR5) 4800 MHz RAM modules on Core i3 12100 (i.e., 4 P-cores and eight threads), those of 4th generation of Double Data Rate (DDR4) 2133 MHz RAM modules on Core i5 12500 (i.e., 6 P-cores and 12 threads), and those of DDR4 2133 MHz RAM modules on Core i7 12700 (i.e., 8 P-cores, 4 E-cores, and 20 threads). We created five baseline ransomware detectors to compensate for variations in the deep-learning models for each subgroup. The detectors predicted a class of the features χ, χ_s, and χ_m of each of the subgroups **(2-1)**, **(2-2)**, and **(2-3)** five times, and the F scores were recorded.

Fig. 12 shows box plots of the F scores when we tested the baseline ransomware detector trained using 100% of the group **(1)** (i.e., the access patterns collected on the machine using DDR4 RAM modules with the sixth generation Intel CPU and with 12th generation Intel CPU) to predict two classes (i.e., ransomware or benign application). The detector trained using both storage and memory features χ of the group **(1)** (i.e., DDR4 RAM modules) was able to detect the storage and memory features χ of the subgroup **(2-1)**, which were collected on DDR5 RAM modules and the same CPU (i.e., the 12th generation Intel Core i3 12100), at the median F score of 0.967 (i.e., the left-most box and whisker) in binary classification; it was close to the F score 0.978 of the baseline ransomware detector at T_d = 30 s and T_{window} = 0.1 s in Fig. 7(a). Thus, we confirmed that the detector trained using DDR4 RAM modules could detect ransomware executed on the newer DDR5 RAM modules with only a 1.1% decrease in F score. In Fig. 12, the third box and whisker from the left shows the median F score of 0.920 when we tested the ransomware detector trained using only the memory features χ_m of the group **(1)** to predict a class using only memory features χ_m of the subgroup **(2-1)**; it was almost identical to the F score 0.926 of the baseline ransomware detector at T_d = 30 s and T_{window} = 0.1 s in Fig. 7(c). In contrast, the median F score was 0.891 when we tested the ransomware detector trained using only storage features χ_s to predict a class of the subgroup **(2-1)**; it was 7.9% lower than the F score 0.970 of the baseline ransomware detector at T_d = 30 s and T_{window} = 0.1 s in Fig. 7(b). Although we conducted experiments in limited hardware settings, we confirmed that storage features χ_s are more sensitive than memory features χ_m to the changes in the generations of DDR RAM modules. Please note that we changed the generation of DDR RAM modules, and it required the change in the motherboard with the latest chipset; we believe that the change to the latest motherboard contributed to a decrease in the F score when we tested the ransomware detector using only storage features χ_s.

We next examined the impacts on the number of CPU cores in the same 12th-generation Intel CPUs. Our baseline ransomware detector was trained using 100% of the group **(1)**, including the access patterns of both the sixth generation Intel CPU (i.e., Celeron G3920 that has two cores and provides two threads) and the 12th generation Intel CPU (i.e., Core i3 12100 CPU that has 4 P-cores and provides eight threads). Then, we tested the baseline ransomware detector to predict a class of the access patterns of the subgroup **(2-2)** of Core i5 12500 CPU that has 6 P-cores and provides 12 threads and the subgroup **(2-3)** of Core i7 12700 CPU that has both 8 P-cores and 4 E-cores that provides 20 threads in total. In this experiment, we examined how the baseline ransomware detector trained using the lower number of CPU cores can detect access patterns executed on the more significant number of CPU cores. In Fig. 12, the median F score of Core i5 and Core i7 using both storage and memory access patterns χ was 0.959 and 0.911, respectively (i.e., the fourth and seventh boxes and whiskers from the left); it was 1.9% and 6.7% lower than the F score 0.978 of the baseline ransomware detector at T_d = 30 s and T_{window} = 0.1 s in Fig. 7(a), respectively. The results indicate that the detection rate improves when the number of CPU cores on the test dataset is close to those on the training dataset. The median F score on which we tested the baseline ransomware detector to predict a class using only memory access patterns χ_m on Core i7 was 0.614, the lowest F score in the tests. The result indicates that the number of CPU cores affects memory

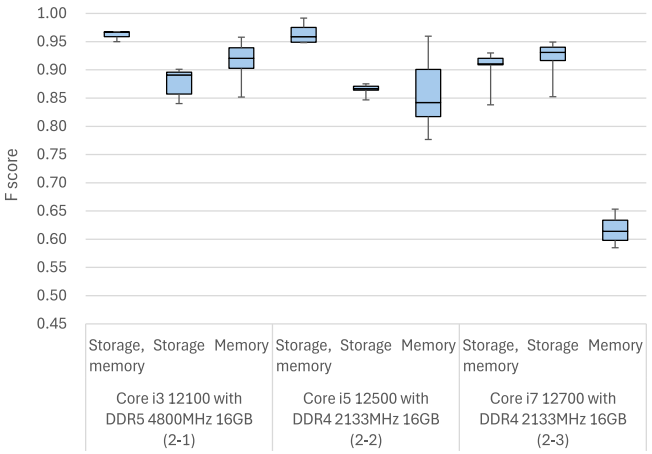


Fig. 12. Box plots of F scores of binary classification when the baseline ransomware detectors predict a class of the access patterns of subgroup **(2-1)**, **(2-2)**, and **(2-3)**; the baseline ransomware detector was trained using 100% of the group **(1)**.

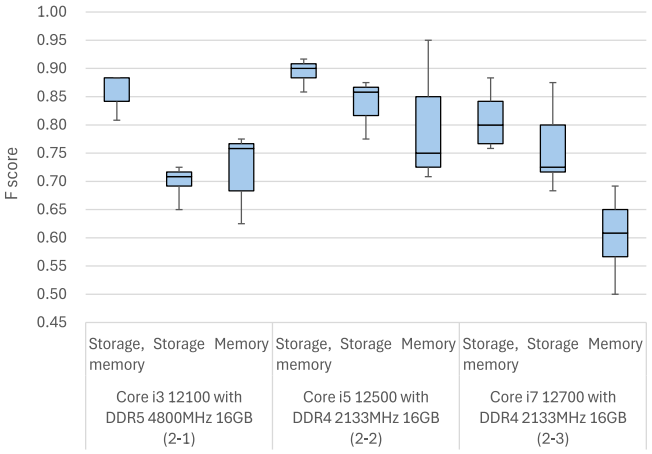


Fig. 13. Box plots of F scores of 12-classes classification when the baseline ransomware detectors predict a class of the access patterns of subgroup **(2-1)**, **(2-2)**, and **(2-3)**; the baseline ransomware detector was trained using 100% of the group **(1)**.

features than storage features. In particular, Core i7 has both P-cores and E-cores; we believe this architecture change affected the detection rates when we used only memory features χ_m.

Fig. 13 shows box plots of F scores in 12-classes classification when we tested the baseline ransomware detector trained using 100% of the group **(1)** to predict a class (i.e., ransomware class names or benign application class names) of the subgroup **(2-1)**, **(2-2)**, and **(2-3)** shown in Table 2. The median F scores were 0.883, 0.900, and 0.800, respectively, when we tested the baseline ransomware detector to predict a class using both storage and memory features χ in DDR5 RAM modules, Core i5, and Core i7, respectively (i.e., the first, fourth, and seventh boxes and whiskers from the left in Fig. 13); they were 5.9%, 4.2%, and 14.2% lower than the F score 0.942 of the baseline ransomware detector at T_d = 30 s and T_{window} = 0.1 s in Fig. 8(a), respectively. In this experiment, we confirmed the following similar trends in 12-classes classification with those in binary classification: (1) the changes in the generation of DDR RAM modules and the motherboard affected the median F score of storage features than that of memory features (i.e., the second and third boxes and whiskers from the left in Fig. 13), and (2) the changes in the number of CPU cores on the same motherboard affected memory features than storage features

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes	Recall in 12 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Conti #7	90	20	55	15	0	30	15	5	0	0	5	15	0.84	0.36
Conti #6	45	0	15	0	0	10	15	40	75	50	0	0	0.28	0.18
Conti #5	50	50	20	15	5	0	10	25	0	0	0	75	0.56	0.20
Conti #4	40	50	35	0	0	20	50	15	0	0	0	40	0.58	0.16
Conti #3	50	15	25	15	0	55	35	15	0	0	5	35	0.64	0.20
Conti #2	40	20	40	5	0	55	25	20	0	0	0	45	0.64	0.16
Conti #1	15	10	45	0	0	5	75	15	0	0	5	80	0.30	0.06

(a) Memory and storage features, χ

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes	Recall in 12 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Conti #7	30	40	50	0	0	25	50	0	0	0	15	40	0.58	0.12
Conti #6	0	0	0	0	0	0	5	50	40	155	0	0	0.00	0.00
Conti #5	25	190	10	0	0	5	10	0	0	0	0	10	0.92	0.10
Conti #4	15	130	15	0	0	10	65	0	0	0	0	15	0.68	0.06
Conti #3	20	65	20	0	0	30	80	0	0	0	15	20	0.54	0.08
Conti #2	35	55	40	0	0	30	50	0	0	0	5	35	0.64	0.14
Conti #1	5	215	15	0	0	5	0	0	0	0	0	10	0.96	0.02

(b) Storage features, χ_s

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes	Recall in 12 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Conti #7	20	125	5	20	5	30	35	0	0	0	10	0	0.82	0.08
Conti #6	55	0	5	10	0	20	15	25	50	65	0	5	0.36	0.22
Conti #5	50	30	30	0	0	10	50	5	0	0	0	75	0.48	0.20
Conti #4	70	50	30	0	0	50	20	5	0	0	0	25	0.80	0.28
Conti #3	75	75	10	0	0	45	35	5	0	0	0	5	0.82	0.30
Conti #2	75	90	35	0	0	40	10	0	0	0	0	0	0.96	0.30
Conti #1	0	0	30	0	0	160	5	20	0	0	10	25	0.76	0.00

(c) Memory features, χ_m

Fig. 14. Confusion matrices and recalls when the baseline ransomware detector predicts a class using the access patterns of the group **(3)**, Conti ransomware variants. The baseline ransomware detector was trained using 100% of the group **(1)**.

(i.e., the fifth, sixth, eighth, and ninth boxes and whiskers from the left in Fig. 13).

6.4. Detection performance on ransomware variants

Next, we examined how the baseline ransomware detector could detect ransomware variants. We employed the seven Conti variants submitted to VirusTotal between 2020 and 2023, shown in Table 6. Source codes of Conti ransomware leaked on February 2022 (Alzahrani et al., 2022); we, therefore, can predict some behaviors on the latest Conti variants, including the state-of-the-art obfuscation technique to bypass API-based detection and ChaCha20-based high-speed file encryption using multi threads. The baseline ransomware detector was trained using 100% of the group **(1)** in Table 2. Then, we tested the baseline ransomware detector to predict a class of the features χ, χ_s, and χ_m of the seven Conti ransomware variants of the group **(3)** in Table 2 (i.e., $D_{test} = \{\textbf{(3)}\}$ and $D_{train} = \{\textbf{(1)}\}$).

Figs. 14(a), 14(b) and 14(c) show the confusion matrices and recalls when we trained the baseline ransomware detector using both storage and memory features χ, only storage features χ_s, and only memory features χ_m. The mean recalls in binary classification were 0.55, 0.62, and 0.71 when we used both storage and memory features χ, only storage features χ_s, and only memory features χ_m, respectively; those of 12-classes classification were 0.19, 0.07, and 0.20, respectively. Recalls were calculated in Eq. (5).

The highest recall in binary classification (i.e., ransomware or benign application) when we used both storage and memory χ was 0.84 in the prediction of the Conti ransomware variant #7; it was the latest Conti variant submitted to VirusTotal in August 2023. The baseline ransomware detector was trained using Conti ransomware submitted in June 2022, as shown in Table 4. The Conti variant #6 has the closest submitted date of March 2022 to the Conti sample that the baseline ransomware detector used in training; however, the recall of the Conti variant #6 was the lowest at 0.28 in binary classification. The oldest Conti variant #1 was submitted to VirusTotal in October 2020; the recall was the second lowest at 0.30 in binary classification.

The highest recall of 0.96 in Fig. 14(b) was on the Conti variant #1 submitted to VirusTotal in October 2020. On the other hand, the highest recall of 0.96 in Fig. 14(c) was on the Conti variant #2 submitted to VirusTotal in November 2020. In the experiments, we confirmed that a single detector was not able to detect ransomware variants correctly; however, the ensemble approach (e.g., the combination of a detector trained using only storage features χ_s and a detector trained using only memory features χ_m) might be effective in detecting ransomware variants that have features similar to only one side of the two features. We could not observe the temporal or evolutionary trends in detecting ransomware variants in the experiments.

6.5. Detection performance in more realistic usage

We tested our baseline ransomware detector in more realistic usage, which executes multiple programs simultaneously. The ransomware detector was trained using 100% of the group **(1)**. Then, we tested the baseline ransomware detector to predict a class of the access patterns of the group **(4)** in Table 2; the access patterns were collected using the methods shown in Table 7. We used the Conti and REvil ransomware samples in Table 4 (i.e., $D_{test} = \{\textbf{(4)}\}$ and $D_{train} = \{\textbf{(1)}\}$).

Fig. 15 shows the three confusion matrices and recalls of the tested group **(4)**. Recalls were calculated in Eq. (5). Please note that the

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	
AESCrypt and Conti	10	0	5	0	0	90	135	5	0	0	5	0	0.42
SDelete and Conti	0	0	5	0	15	0	10	35	0	5	175	5	0.08
Zip and Conti	25	0	5	0	0	15	110	25	0	0	0	70	0.18
Firefox and Conti	0	0	0	0	5	0	115	130	0	0	0	0	0.02
AESCrypt and REvil	0	0	0	0	0	0	110	30	0	0	0	110	0.00
SDelete and REvil	0	0	0	0	0	0	0	0	0	0	250	0	0.00
Zip and REvil	0	0	15	0	0	0	0	0	0	0	0	235	0.06
Firefox and REvil	5	0	10	235	0	0	0	0	0	0	0	0	1.00
Office and REvil	0	0	0	250	0	0	0	0	0	0	0	0	1.00
Firefox, Office, and REvil	5	5	20	180	10	0	0	25	5	0	0	0	0.88

(a) Memory and storage features, χ

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	
AESCrypt and Conti	100	15	0	0	0	55	50	25	0	0	0	5	0.68
SDelete and Conti	0	0	15	0	0	50	25	5	0	25	130	0	0.26
Zip and Conti	50	5	0	0	0	10	35	60	0	0	0	90	0.26
Firefox and Conti	10	0	0	0	0	20	120	100	0	0	0	0	0.12
AESCrypt and REvil	10	5	30	0	0	25	70	20	0	0	0	90	0.28
SDelete and REvil	0	0	0	0	0	0	0	0	0	0	250	0	0.00
Zip and REvil	0	5	20	60	0	0	0	5	0	0	0	160	0.34
Firefox and REvil	0	0	0	125	20	0	15	20	25	0	0	45	0.58
Office and REvil	0	0	0	200	5	0	0	0	45	0	0	0	0.82
Firefox, Office, and REvil	5	0	40	75	75	0	5	25	20	5	0	0	0.78

(b) Storage features, χ_s

Predicted class

Actual class	Ransomware						Benign						Recall in 2 classes
	Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	
AESCrypt and Conti	0	0	10	0	5	75	20	30	0	0	5	105	0.36
SDelete and Conti	5	5	5	0	85	0	0	35	10	10	95	0	0.40
Zip and Conti	0	0	0	0	25	55	10	15	0	0	0	145	0.32
Firefox and Conti	10	0	0	0	30	50	40	110	10	0	0	0	0.36
AESCrypt and REvil	10	0	15	0	0	40	65	65	0	0	0	55	0.26
SDelete and REvil	10	10	0	0	0	0	0	0	0	0	230	0	0.08
Zip and REvil	0	0	15	0	0	120	5	5	0	0	0	105	0.54
Firefox and REvil	5	95	0	60	0	0	20	10	0	0	60	0	0.64
Office and REvil	0	170	0	75	0	0	5	0	0	0	0	0	0.98
Firefox, Office, and REvil	0	40	10	65	0	0	80	25	0	0	30	0	0.46

(c) Memory features, χ_m

Fig. 15. Confusion matrices and recalls when the baseline ransomware detector predicts a class using the access patterns of the group **(4)**, simultaneous execution of ransomware and benign applications. The baseline ransomware detector was trained using 100% of the group **(1)**.

Conti ransomware sample stopped all the processes of Office applications (i.e., Word, Excel, and PowerPoint programs) immediately after the Conti ransomware was executed; therefore, we could not collect access patterns when Conti and Office were executed simultaneously. Fig. 15(a) shows the confusion matrix of the baseline ransomware detector using both memory and storage features χ. REvil ransomware was precisely detected at the recalls between 0.88 and 1.00 when Firefox web browser and Office applications were executed simultaneously (i.e., the bottom three rows). We can see the improvement in recalls by combining storage features χ_s (Fig. 15(b)) and memory features χ_m (Fig. 15(c)) when we executed REvil ransomware with Firefox web browser and Office applications simultaneously. In Fig. 15(b), we used only storage features χ_s; the mean recall of the bottom three rows was 0.73. In Fig. 15(c), we used only memory features χ_m; the mean recall of the bottom three rows was 0.69. However, when we used storage and memory features χ (Fig. 15(a)), the mean recall of the bottom three rows significantly improved to 0.96. Although our previous dataset RanSAP (Hirano et al., 2022) contained only storage features and was not able to detect simultaneous execution of ransomware and Office applications, the new RanSMAP dataset presented in this paper that contains both storage and memory features was able to detect REvil ransomware even when Office applications and Firefox web browser were executed simultaneously. However, detecting Conti ransomware with the Firefox web browser was more difficult than REvil ransomware with the Firefox web browser (i.e., the fourth and eighth rows in Figs. 15(a), 15(b) and 15(c)). We confirmed that the difficulty in detecting ransomware depends on the behaviors of ransomware samples.

On the other hand, most of the REvil ransomware executed with AESCrypt, SDelete, and Zip (i.e., the fifth, sixth, and seventh rows from the top) were classified as AESCrypt, SDelete, and Zip, respectively, in all the features χ, χ_s, and χ_m (i.e., Figs. 15(a), 15(b) and 15(c)). In the experiment, we confirmed that the simultaneous executions of ransomware with AESCrypt, SDelete, and Zip are more challenging to detect than those of ransomware with Firefox and Office applications; we observed similar results on Conti ransomware (i.e., the first three rows in Figs. 15(a), 15(b) and 15(c)). The result indicates that simultaneous execution of ransomware samples with encryption, secure delete, and compression programs can bypass behavioral-based ransomware detection systems even if we use the RanSMAP dataset, which consists of storage and memory features.

7. Discussion

In this section, we present some state-of-the-art research to compare to our behavioral feature-based ransomware detection method, related work and availability of the ransomware behavioral feature dataset, use cases, and limitations of the RanSMAP dataset.

7.1. State-of-the-art

Defense-in-depth. Ma et al. presented a sophisticated data recovery method named *RansomTag* (Ma et al., 2023) by combining SSD firmware and a lightweight hypervisor to implement the defense-in-depth strategy similar to our presented work. They employed BitVisor (Shinagawa et al., 2009), which we used in creating the RanSMAP dataset and ransomware detector. Although *RansomTag* and our presented work employed the same thin hypervisor, the essential difference between them is whether the semantic gap problem is solved. Ma et al. implemented the introspection module in BitVisor to interpret the high-level semantics of the OS layer (i.e., the process and file information of Windows OS). On the other hand, our storage access monitor function in BitVisor collects low-level memory and storage access patterns as they are. From the functionality perspective, our presented work provides a ransomware detection function but a recovery function. From the perspective of the implementation method, our presented system is implemented using only a hypervisor. In contrast, *RansomTag* is implemented using a hypervisor and SSD firmware.

Early detection. Early detection reduces the impacts of ransomware attacks. Roy and Chen presented early detection methods using attention-based Bi-directional LSTM and Conditional Random Fields using host logs (Roy and Chen, 2021). Singh and Tripathy used permissions and API calls to detect Android ransomware in the early stage (Singh and Tripathy, 2024). Their method can detect ransomware attacks before fully executing on the victim host. Although the host logs, permissions, and API calls provide reliable evidence of ransomware deployment, sophisticated attacks can disable detection mechanisms on the OS layer using escalated administrator privilege. In contrast, our ransomware detection method using low-level behavioral features using a thin hypervisor provides an additional protection layer. Our current LSTM-based baseline ransomware detector can detect ransomware's destruction phase in 30 s (i.e., $T_d = 30$ s); however, the latest high-speed ransomware encryption functions require us to detect them in less time. Therefore, we need to reduce the detection time further to mitigate the impact of the destruction phase by improving the design of our preprocessing method and deep-learning model. Under the current system, we can combine a simpler detection method with low computational cost (e.g., conventional signature-based antivirus systems on operating system) with our detection method; our system provides a second defense against attackers who compromised the first defense system (i.e., the signature-based antivirus). This defense-in-depth strategy will improve the security of mission-critical systems.

Lowering impacts of attacks and recovery. Berardi et al. presented another approach to lower the impact of ransomware attacks by flooding files in ransomware attacks (Berardi et al., 2023). Their approach is similar to deception mechanisms, such as a honeypot that provides attackers decoy files on the OS layer to slow down their operations. For example, our thin hypervisor can slow down I/O operations using HyperSlow (Okamura and Oyama, 2012) to produce similar results by Berardi et al.. The recovery of encrypted files is another direction of ransomware research; Yuste and Pastrana revealed the internal structure of Avaddon ransomware and developed the decryption tool (Yuste and Pastrana, 2021). Filiz et al. tested 78 ransomware decryption tools and showed that nearly half failed to recover compromised data satisfactorily (Filiz et al., 2021). In contrast, our current detection system does not provide a recovery function. Our storage access monitoring function sends all the written blocks on SSD to the monitoring machine; our system, therefore, can be used for a data recovery function after ransomware attacks. However, it is not practical for every user to use another monitoring machine for backup purposes; it might be possible to store a certain amount of written blocks in a limited RAM area managed by a hypervisor. To implement the above feature, we need to detect ransomware attacks in the shortest possible time to increase the success rate of file recovery since the memory capacity managed by the hypervisor is limited.

Concept drift and explainable AI. Recent topics of machine-learning-based ransomware detection include the impact of *concept drift*. *Concept drift* is the phenomenon yielded when the data distribution changes over time (Widmer and Kubat, 1996). *Adaptive learning* refers to updating models online during their operation to react *concept drift*. While general adaption methods of concept drift have been studied in detail (Gama et al., 2014), that on evolutionary malware (e.g., malware variants) have not been well studied. Fernando and Komninos presented the novel machine learning classifier to detect evolutionary ransomware by adding drift calibration and decision layer (Fernando and Komninos, 2024). From *concept drift* perspective, the *adaptive learning* is essential to detect the recent ransomware strains, for example, that enhance evasion and stealth capabilities. Commercial antivirus and security products provide frequent updates to counteract the latest attacks. For the same reason, deep-learning-based ransomware detection systems will need frequent updates of deep-learning models. Our current ransomware detector does not provide *adaptive learning* function; we need to monitor and update training data (i.e., the pairs of labels and storage and memory access patterns) that evolve over time. The data distribution evolution is caused by changes in the strategies of ransomware developers (e.g., recent trends are not encryption but exfiltration). Thus, the deep-learning-based detection systems have the same problems (i.e., the need for frequent updates) as signature-based detection systems.

In addition, the explainability of machine-learning-based classifiers helps human experts design better deep-learning models dealing with evolutionary ransomware and its variants (i.e., *concept drift* of the ransomware detector). Gulmez et al., for example, presented a ransomware detection system using Local Interpretable Model-Agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) (Gulmez et al., 2024). When we need *adaptive learning* to counteract *concept drift* problems, we will be able to use these explainable AI tools such as LIME and SHAP to analyze the cause of the *concept drift* (i.e., the strategy changes of ransomware developers) by monitoring the LIME and SHAP values periodically. Since ransomware analysts usually cannot obtain ransomware's source code, they need to perform expensive reverse engineering tasks; the reverse engineering task is time-consuming even if analysts have sufficient experience. The explainable AI and conventional statistics tools have the potential to assist professional analysts by noticing ransomware's behavioral changes using the data distribution of storage and memory access patterns (e.g., a decrease in entropy features might indicate the adoption of evasive attacks).

7.2. Related work and availability of ransomware behavioral feature datasets

Botacin et al. surveyed the state-of-the-art malware research and its pitfalls, and they found that only seven percent of the research papers released their datasets, showing that a small portion of them is reproducible (Botacin et al., 2021); they proposed guidelines to advance the state-of-the-art of the malware research filed, for example, making researchers' datasets publicly available to facilitate research reproducibility. The ransomware datasets can be considered as a subset of the malware datasets. We can categorize the open malware datasets into the following groups: (1) the *primary* dataset consisting of malware samples and its metadata and (2) the *secondary* dataset consisting of data obtained from the primary dataset. The

Table 9
Related work of ransomware detection systems and open behavioral feature dataset availability.

Reference	Behavioral features on OS layer	Entropy	Low-level Storage I/O	Memory accesses	Open dataset	Monitoring layer
RanSMAP	N/A	√	√	√	√	Hypervisor
Singh and Tripathy (2024)	API call					OS
Fernando and Komninos (2024)	API call					OS
Herrera-Silva and Hernández-Álvarez (2023)	Cuckoo JSON report	√			√	OS
Ma et al. (2023)	EPROCSS and IRP structure	√				Hypervisor
McIntosh et al. (2023)	IRP					OS
Hirano et al. (2022)	N/A	√	√		√	Hypervisor
Amer and El-Sappagh (2022)	API and system call					OS
Roy and Chen (2021)	Windows host log					OS
Tang et al. (2020)	System call	√				Hypervisor
Berrueta et al. (2020)	SMB traffic				√	OS
Hull et al. (2019)	Cuckoo JSON report					OS
Hampton et al. (2018)	API call					OS
Shaukat and Ribeiro (2018)	IRP	√				OS
Kharraz and Kirda (2017)	IRP	√				OS
Continella et al. (2016)	IRP	√				OS
Scaife et al. (2016)	IRP	√				OS
Kharaz et al. (2016)	IRP	√				OS

examples of the *primary* dataset are SOREL-20M, including 20 million disarmed malware samples (Harang and Rudd, 2020) and CICMalDroid2020 (Mahdavifar et al., 2020) including 17,341 Android malware executables. The primary dataset also includes malware repositories, such as VirusTotal (Chronicle, 2004), VirusShare (Corvus Forensics, 2007), ANY.RUN (ANY.RUN, 2017), and AndroZoo (Allix et al., 2016). On the other hand, the *secondary* dataset includes static or dynamic features obtained using the *primary* dataset. The RanSMAP dataset is a *secondary* dataset; we collected storage and memory access patterns of ransomware executables obtained from the *primary* dataset, ANY.RUN (ANY.RUN, 2017).

Table 9 shows related work of ransomware detection systems and the availability of open ransomware behavioral feature datasets. Although many papers describe the static analysis of ransomware and malware, this section shows only the availability of the ransomware dynamic feature (i.e., behavioral feature) datasets. We categorized behavioral features into the following groups: API and system calls, sandbox reports, memory artifacts, network artifacts, OS logs, and I/O Request Packet (IRP). We discuss the difference between each behavioral feature group and our presented work, and finally, the availability of the open datasets.

API and system calls. Singh and Tripathy employed permissions and API calls of Android systems to detect ransomware at the pre-operation stage (Singh and Tripathy, 2024). Fernando and Komninos presented a ransomware detection system using API calls that reliably detects and classifies concept drift samples reliably (Fernando and Komninos, 2024). Amer and El-Sappagh presented an Android malware prediction method using API and system call sequences and tested it to detect ransomware samples; they used eight open Android malware feature datasets containing API call sequence, system call sequence, and permissions created between 2016 and 2021 (Amer and El-Sappagh, 2022). Hampton et al. used Windows API sequences collected using Process Monitor on Windows OS (Hampton et al., 2018). Although API-based and system-call-based detectors (Singh and Tripathy, 2024; Fernando and Komninos, 2024; Amer and El-Sappagh, 2022; Hampton et al., 2018) provide better semantic information (e.g., file system and network activities) than our presented work, they cannot provide the defense-in-depth strategy since they can be disabled by attackers who obtained administrative privilege of the operating system. On the other hand, a hypervisor-based ransomware detection system named *RansomSpector* (Tang et al., 2020) is similar to our proposal; *RansomSpector* traps *SYSCALL* instruction of x86 CPUs to intercept system call sequences related to file system and network activities. *RansomSpector* also obtained information on the caller process by interpreting Windows kernel data structure on RAM. The latter feature is vulnerable to attackers that can manipulate kernel structure in RAM using the administrative privilege of the OS. In contrast, our method uses low-level storage and memory access patterns as they are without solving the semantic problem.

Sandbox reports. Herrera-Silva and Hernández-Álvarez presented a dynamic feature dataset for ransomware detection obtained using Cuckoo sandbox (Herrera-Silva and Hernández-Álvarez, 2023). Hull et al. presented an analysis of ransomware deployment methods, including listing and encrypting files, deleting original files, and threatening with ransom notes using Cuckoo sandbox reports (Hull et al., 2019). Although their dataset is collected on the sandbox environments created using a hypervisor, sandbox reports are collected using agent program on the operating system; therefore, sandbox reports is not protected from attackers who obtained administrative privilege of the OS. In contrast, our monitoring system is protected from attackers on the OS since it works on the hypervisor layer. In addition, the presented method does not need any agent programs on the OS and modification of the original OS environment.

Memory artifacts. Ma et al. presented *RansomTag* (Ma et al., 2023), a ransomware detection and recovery mechanism using SSD firmware and hypervisor. *RansomTag* uses the high-level OS semantic information of ransomware (i.e., EPROCESS and I/O Request Packets structure on RAM) to detect ransomware. Their *RansomTag* solved the semantic gap problem by interpreting some kernel structures of the OS in the hypervisor layer. Although high-level semantic information is helpful in reliably detecting ransomware, it can be bypassed when attackers obtain administrator privileges on the OS and manipulate the kernel structure on RAM. In contrast, our method has some resilience against such manipulation attacks of kernel structure on RAM since we intentionally use low-level storage and memory access patterns as they are.

OS logs. Roy and Chen presented a ransomware detection method using Windows Logging Service (Roy and Chen, 2021); they created a ransomware detection model based on Bi-directional Long Short-Term Memory (BiLSTM) to process the OS log files. OS and application logging services are useful for administrative and forensic purposes; however, the OS logs can be modified or deleted by attackers who obtain the administrative privilege of the OS.

Network artifacts. Berrueta et al. presented an open ransomware behavioral feature dataset containing ransomware's file operations; the file operations were collected on the shared network volume of a remote file server by capturing Server Message Block (SMB) protocol packets (Berrueta et al., 2020). Their method (i.e., capturing SMB packets to detect file system operations) cannot be applied to ransomware attacks on local storage devices. In contrast, our present work can detect ransomware on local storage devices. We, however, need to capture SMB packets to detect ransomware operations on remote

volumes. In our future work, we plan to extend our thin hypervisor to monitor network traffic to counter such remote attacks.

I/O Request Packet, IRP. Many ransomware detection systems presented between 2016 and 2018, including *UNVEIL* (Kharaz et al., 2016) employ I/O Request Packets (IRP) obtained using Windows Filesystem Minifilter Driver framework (Microsoft, 2021), which is used in AntiVirus software. A file system filter driver, or minifilter, of Windows intercepts I/O requests targeted at a file system driver to extend the functionality of the original driver. Therefore, Windows' minifilter drivers can be used in antivirus, backup, and encryption products. Since the Minifilter framework is executed in the OS layer, it can be compromised by attackers who obtain the OS's administrative privileges. In recent research, McIntosh et al. presented a ransomware detector using IRP packets and their concept of user-centric and program-centric access control to McIntosh et al. (2023). The IRP structures of Windows OS include high-level semantic information, such as file system operations, file object, process, and thread information. AntiVirus products can use IRP information to detect malware reliably. In contrast, our presented work does not use IRP information since our system does not interpret OS-specific high-level semantic information. Although low-level storage access patterns do not have complete information on IRP, Hirano et al. showed that a thin hypervisor can obtain storage access patterns that have almost identical behavioral features to those obtained using IRP packets (Hirano et al., 2022). However, since our presented work does not distinguish I/O operations between processes, we need to extend our monitoring system of system-wide storage and memory access patterns to distinguish per-process I/O operations for more precise detection.

Availability of dataset. Finally, we discuss the availability of the open dataset. Herrera-Silva and Hernández-Álvarez released a dynamic feature dataset of ransomware obtained using Cuckoo sandbox (Herrera-Silva and Hernández-Álvarez, 2023). Berrueta et al. presented an open ransomware behavioral feature dataset containing packet capture files; they contain ransomware activities of SMB packets to the shared network volume on a remote file server (Berrueta et al., 2020). On the other hand, RanSAP (Hirano et al., 2022), our previous behavioral feature dataset, includes storage access patterns obtained using a thin hypervisor. The RanSMAP dataset, presented in this paper, is the successor dataset of RanSAP and one of the few open datasets of ransomware behavioral features obtained using a thin hypervisor. Our datasets, RanSMAP, are publicly available in GitHub and Kaggle (Hirano, 2024a,b).

7.3. Use cases

The primary use case of our ransomware detection method using the RanSMAP dataset is for a building block of the defense-in-depth ransomware detection and prevention systems. Thus, our system is intended to supplement existing antivirus and ransomware detection systems. As we reviewed in the previous section, ransomware detection systems have evolved to detect various behavioral features. The primary ransomware detection method shown in Table 9 employs dynamic features of OS semantics, including file system activities obtained using I/O Request Packets (IRP), API calls, system calls, host logs, process information, and sandbox reports (e.g., Cuckoo JSON report). If sophisticated attackers gain administrator privileges on a host, they can stop ransomware detection services on the OS layer. Moreover, they can also manipulate process and kernel information on RAM, which includes API and system call traces that can be used to detect ransomware. Manipulated OS semantic information on RAM can be used to bypass the detection function even when the monitoring mechanism is executed on hypervisor layer (Ma et al., 2023; Tang et al., 2020). Our approach (i.e., additional protection of the hypervisor layer that uses low-level storage and memory access patterns without OS semantics) can increase resilience against such sophisticated evasion attacks when the first protection mechanism on the OS layer is compromised.

Our dataset includes Conti ransomware; the Conti sample we used is also known as BlueSky, a Conti v3 variant[1] (Palo Alto Networks, 2022); it has API obfuscation mechanisms of indirect API calls to prevent API-based detection in OS layer. Since BlueSky's API calls are concealed using API hashing, the first defense against ransomware attacks (e.g., static analysis of binary programs before execution) cannot search signatures of API call sequences implemented in ransomware binary without knowledge of its evasion techniques. However, we successfully detected the Conti sample at an F score of 94.3% in 12 classes, as shown in Fig. B.1. The experimental results showed that our method, which does not depend on API-based behavioral artifacts, can detect ransomware while reducing the risk of evasion attacks using specific OS-level features such as API calls. Our detection method uses only low-level storage and memory access patterns obtained using a thin hypervisor and, therefore, supports defense-in-depth systems to prevent OS-level evasion attacks.

7.4. Limitation

Semantic gap problem. We first presented the following research question: Can low-level storage and memory access patterns obtained using hypervisor discriminate ransomware from benign applications without solving the semantic gap problem? The problem arises because the hypervisor does not have a complete view of the guest operating system's activities, such as high-level activities using the file system, process, and system call abstractions. The experimental results showed that we could detect ransomware without solving the semantic gap problem under certain conditions. For example, we can detect malicious behaviors when REvil ransomware is executed simultaneously with Firefox and Office applications, as shown in Fig. 15. However, in the same experiment, we also confirmed that the simultaneous execution of ransomware and the specific benign applications (i.e., AESCrypt, SDelete, Zip) make detection challenging since the specific benign behaviors mask the ransomware's malicious behaviors. The current thin-hypervisor-based monitoring system collects *system-wide* storage and memory access patterns; our future work will need to consider context switching to obtain *process-specific* storage and memory access patterns to prevent masking ransomware's malicious activities using benign applications. This improvement is challenging since we need to solve the semantic gap problem on process abstraction of the OS layer on the hypervisor layer; however, this improvement will be necessary to decrease false negatives and, more importantly, to prevent evasion attacks that mask malicious behaviors.

Dataset contents and analysis. The RanSMAP dataset includes six ransomware samples and seven Conti ransomware samples; to reach a generally extrapolated conclusion, we need more access patterns of additional ransomware samples, those of benign applications, and those of simultaneous executions. However, dataset construction is time-consuming; therefore, we need some automation methods to construct larger datasets. Since our thin hypervisor, used to collect storage and memory access patterns, is a bare-metal Type-I hypervisor, automation is more challenging than Type-II hypervisor, which runs on the host OS. In addition, our experimental results do not include a detailed per-sample analysis of ransomware detection. The experimental results shown in Fig. B.1 were obtained using the baseline ransomware detector trained using 80% of the entire dataset group (1) and tested using 20% of the entire dataset group (1); researchers who use the RanSMAP dataset, for example, can analyze the detection performance of specific ransomware sample (e.g., Conti) using a detector trained using other ransomware samples without Conti. The experimental results of the Conti variants shown in Fig. 14, on the other hand, were obtained using the detector trained using 100% of the entire dataset group

[1] SHA256 hash is e75717be1633b5e3602827dc3b5788ff691dd325b0edd2d0d9ddcee29de364f.

Actual class		Predicted class: Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	F score in 12 classes	F score in 2 classes
Ransomware	Conti	222	0	6	0	0	0	4	0	2	0	0	0	0.943	0.977
	Darkside	0	233	2	0	0	0	1	0	2	0	0	0	0.975	
	LockBit	5	5	196	0	1	0	0	1	0	1	1	3	0.907	
	REvil	0	0	0	238	0	0	1	1	0	1	0	0	0.992	
	Ryuk	2	0	2	1	214	0	2	6	1	0	1	0	0.951	
	WannaCry	0	1	0	0	0	244	2	2	3	1	0	0	0.978	
Benign	AESCrypt	1	1	3	0	0	1	230	1	0	1	0	4	0.935	0.978
	Firefox	0	0	4	0	3	0	2	239	1	7	1	1	0.919	
	Idle	4	0	4	0	3	0	1	2	229	14	0	1	0.895	
	Office	3	0	2	0	0	1	0	9	16	209	0	0	0.882	
	SDelete	0	0	0	0	0	0	0	0	0	0	230	0	0.994	
	Zip	0	0	0	0	0	0	7	1	0	0	0	236	0.965	

(a) Memory and storage features, χ

Actual class		Predicted class: Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	F score in 12 classes	F score in 2 classes
Ransomware	Conti	235	1	6	0	1	1	4	3	0	0	0	1	0.931	0.972
	Darkside	4	206	4	0	0	0	0	0	1	1	0	0	0.967	
	LockBit	7	1	226	0	2	0	0	5	1	1	1	3	0.908	
	REvil	0	0	2	245	0	0	0	0	0	0	0	1	0.982	
	Ryuk	2	0	3	0	218	0	0	5	1	2	0	0	0.934	
	WannaCry	0	0	3	0	1	236	9	0	0	0	0	2	0.954	
Benign	AESCrypt	3	1	2	3	1	5	187	5	2	0	1	13	0.835	0.971
	Firefox	0	0	2	0	7	0	7	222	8	9	1	1	0.857	
	Idle	1	0	1	3	5	1	8	7	209	14	1	4	0.836	
	Office	1	0	1	0	1	0	1	12	24	197	0	1	0.853	
	SDelete	0	0	0	0	0	0	2	0	0	0	230	0	0.987	
	Zip	0	1	1	0	0	1	7	2	0	0	0	219	0.920	

(b) Storage features, χ_s

Actual class		Predicted class: Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip	F score in 12 classes	F score in 2 classes
Ransomware	Conti	201	13	12	0	1	0	1	0	1	1	1	1	0.843	0.957
	Darkside	7	205	9	3	0	0	2	0	1	0	5	3	0.856	
	LockBit	21	11	189	1	3	2	2	6	2	1	5	2	0.786	
	REvil	3	3	1	227	0	0	1	0	0	0	1	0	0.958	
	Ryuk	0	1	1	1	221	0	1	2	1	1	4	0	0.955	
	WannaCry	1	0	6	0	0	239	5	1	0	0	5	8	0.928	
Benign	AESCrypt	1	2	1	2	0	0	213	6	1	0	0	11	0.880	0.956
	Firefox	0	0	2	1	1	0	1	202	4	8	3	1	0.886	
	Idle	2	1	1	0	1	0	3	1	203	14	0	3	0.885	
	Office	4	0	2	0	1	3	1	8	17	203	0	0	0.869	
	SDelete	5	8	12	2	2	1	1	1	0	0	219	0	0.887	
	Zip	0	0	0	1	0	5	16	6	0	0	0	227	0.888	

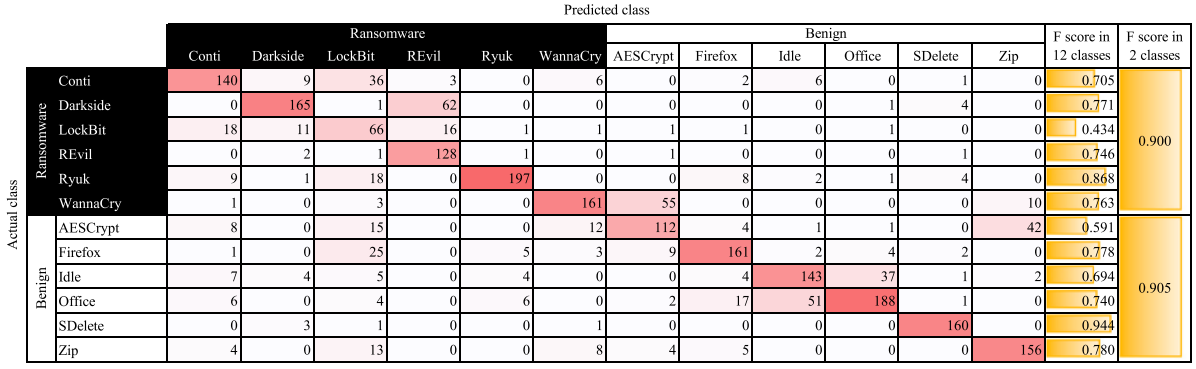
(c) Memory features, χ_m

Fig. B.1. Confusion matrices and F scores of the baseline ransomware detectors; we trained them using randomly selected 80% of the group **(1)** and predicted a class using the remaining 20% of the group **(1)**.

(1); researchers who use the RanSMAP dataset can also analyze the performance of a detector trained using ransomware without Conti. Although this paper does not provide these detailed per-sample analysis results because of limited space, they will show deeper insights into ransomware behavioral features and the similarity between ransomware strains.

Evasion attacks. One of the latest topics in malware detection systems is evasion techniques. Amer et al. presented the malware detection approach through behavioral fusion of API call sequences by integrating statistical, contextual, and graph mining features over API call sequences to detect malware variants that change API call sequences to evade detection (Amer et al., 2021). As malware and ransomware detection systems using behavioral features increased, many evasion techniques on them have been proposed; Witte presented a technique to bypass a malware detection system using behavioral features by imitating benign user activities to mask harmful actions; their *Phantom Ransomware* was able to bypass 12 commercial antivirus products (Witte, 2020). Zhou et al., on the other hand, presented a novel evasion technique on I/O-based ransomware detection systems (Zhou et al., 2023); their imitation-based ransomware attack mimics behaviors of benign programs to disguise its encryption tasks. The attack first collects benign I/O behaviors using Windows IRPs and creates a behavior template; then, it executes encryption tasks while benign file I/O requests are produced. Their imitation-based attacks successfully evade the three major commercial antivirus products and three research-based ransomware detection systems using IRP features (Kharaz et al., 2016; Continella et al., 2016; Kharraz and Kirda, 2017). Diamantopolous et al. presented state-of-the-art evasion attacks on the I/O-based ransomware detection systems inspired by our previous RanSAP dataset (Diamantopolous et al., 2024; Hirano et al., 2022); their highly configurable ransomware emulator named *WannaLaugh* can adjust ransomware storage access patterns to evade the detection system. The configurable parameters include encryption algorithms and modes, encryption speed, percentage of encryption in files, and multi-threaded encryption.

For example, an imitation-based ransomware attack presented by Zhou et al. masks malicious file access patterns of ransomware using



Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	140	9	36	3	0	6	0	2	6	0	1	0	0.705	0.900
	Darkside	0	165	1	62	0	0	0	0	0	1	4	0	0.771	
	LockBit	18	11	66	16	1	1	1	1	0	1	0	0	0.434	
	REvil	0	2	1	128	1	0	1	0	0	0	1	0	0.746	
	Ryuk	9	1	18	0	197	0	0	8	2	1	4	0	0.868	
	WannaCry	1	0	3	0	0	161	55	0	0	0	0	10	0.763	
Benign	AESCrypt	8	0	15	0	0	12	112	4	1	1	0	42	0.591	0.905
	Firefox	1	0	25	0	5	3	9	161	2	4	2	0	0.778	
	Idle	7	4	5	0	4	0	0	4	143	37	1	2	0.694	
	Office	6	0	4	0	6	0	2	17	51	188	1	0	0.740	
	SDelete	0	3	1	0	0	1	0	0	0	0	160	0	0.944	
	Zip	4	0	13	0	0	8	4	5	0	0	0	156	0.780	

(a) Memory and storage features, χ

Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	112	6	54	2	1	3	3	1	1	0	0	1	0.597	0.876
	Darkside	2	186	5	9	0	1	0	0	0	0	0	2	0.890	
	LockBit	25	14	70	1	0	6	5	1	0	0	1	18	0.411	
	REvil	1	0	2	164	0	0	0	2	0	0	0	5	0.882	
	Ryuk	4	0	6	0	138	0	1	6	2	3	0	0	0.784	
	WannaCry	10	0	3	0	0	192	43	1	0	0	3	1	0.828	
Benign	AESCrypt	32	1	9	0	1	7	123	31	3	1	0	35	0.566	0.884
	Firefox	1	0	41	1	30	1	3	103	17	13	3	0	0.516	
	Idle	3	3	2	0	7	0	8	19	140	17	0	0	0.681	
	Office	0	0	0	0	14	0	1	17	49	164	0	1	0.739	
	SDelete	0	0	0	0	0	1	1	0	0	0	195	0	0.977	
	Zip	1	3	8	21	1	0	4	5	0	0	0	142	0.728	

(b) Storage features, χ_s

Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	125	70	29	27	0	0	0	1	2	0	9	0	0.552	0.844
	Darkside	11	126	12	73	0	0	0	0	3	2	32	0	0.516	
	LockBit	30	13	80	19	0	0	0	0	0	1	3	0	0.440	
	REvil	1	9	0	57	0	1	2	1	5	0	0	0	0.418	
	Ryuk	1	1	26	0	148	0	1	9	6	1	18	0	0.789	
	WannaCry	4	0	11	0	0	102	16	3	0	1	0	13	0.567	
Benign	AESCrypt	4	1	6	0	0	17	107	5	6	2	0	29	0.578	0.855
	Firefox	2	1	15	0	2	5	6	134	2	2	5	2	0.766	
	Idle	1	1	3	0	1	0	2	1	141	31	1	0	0.709	
	Office	3	0	5	0	11	0	1	11	51	161	1	2	0.720	
	SDelete	6	6	26	21	1	0	0	4	0	0	138	0	0.675	
	Zip	2	1	5	0	1	85	58	5	0	0	0	155	0.604	

(c) Memory features, χ_m

Fig. C.1. Confusion matrices and F scores when we trained the ransomware detectors using 100% of the group **(1-1)**, Celeron G3920 CPU, and predicted a class using the randomly selected 40% of the group **(1-2)**, Core i3 12100 CPU.

benign file access patterns of legitimate applications to evade entropy-based ransomware detectors (Zhou et al., 2023). On the other hand, *WannaLaugh* presented by Diamantopolous et al. can specify the area of the file to be encrypted (e.g., first few bytes, repeated segments, last few bytes) (Diamantopolous et al., 2024); it can evade entropy-based ransomware detectors by lowering entropy values. As described above, many evasion techniques have been presented to counter entropy-based ransomware detection systems; therefore, we need to create a more resilient defense system that can prevent these state-of-the-art evasion attacks. In addition, recent ransomware attacks exfiltrate data from victim computers without performing encryption. We recognize that detecting exfiltration is more challenging than detecting encryption since exfiltration does not have significant behavioral features compared to encryption. The current RanSMAP dataset includes only storage and memory access patterns to detect file encryption operations of ransomware; future extended datasets will need network features to detect ransomware's exfiltration operations.

Vulnerabilities and stealthiness of hypervisor. Although we employed a thin hypervisor as an additional protection layer in our defense-in-depth approach, the hypervisor can be an attractive attack surface; a popular commercial hypervisor, VMware ESXi, for example, had many vulnerabilities (e.g., heap-overflow issue of CVE-2021-21974) that enables attackers to execute remote command execution on virtualization infrastructure. We, therefore, obviously need to be aware of vulnerabilities of hypervisor; Schumilo et al. presented a hypervisor fuzzing technique to detect hypervisor vulnerabilities (Schumilo et al., 2021). Moreover, we need to monitor ransomware behavior from a thin hypervisor layer as stealthy as possible since attackers can stop ransomware operations when they detect the hypervisor-based monitoring system. Zhang et al. presented the method to detect the existence of hardware-assisted virtualization programs (e.g., hypervisors using Intel VMX extensions) using low-level inconspicuous hardware behavioral features collected from Translation Lookaside Buffer (TLB), Last-Level Cache (LLC), and Level-1 Data (L1D) caches.

Problems on virtual machine introspection. In addition, hypervisor-based virtual machine introspection techniques, including our approach, have inconsistency problems of memory data from the OS's

Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	115	1	31	0	1	0	1	1	2	11	1	4	0.610	0.934
	Darkside	60	200	82	1	0	0	2	0	3	0	0	1	0.718	
	LockBit	21	2	69	1	6	0	0	5	3	1	0	2	0.465	
	REvil	2	3	1	183	0	0	1	0	1	0	0	1	0.956	
	Ryuk	0	0	0	6	184	1	0	22	9	33	2	0	0.795	
	WannaCry	0	0	1	0	0	176	7	0	0	0	1	1	0.926	
Benign	AESCrypt	0	0	0	0	0	8	105	1	3	0	1	2	0.654	0.930
	Firefox	0	0	2	0	8	1	5	154	19	5	0	6	0.753	
	Idle	10	0	1	0	6	0	1	1	132	44	0	0	0.673	
	Office	1	0	0	0	1	4	1	16	21	111	0	0	0.615	
	SDelete	0	1	0	0	0	0	3	3	2	0	192	0	0.965	
	Zip	0	1	0	0	0	4	75	6	2	1	0	178	0.771	

(a) Memory and storage features, χ

Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	156	16	19	0	8	0	6	8	2	1	0	3	0.729	0.909
	Darkside	13	171	47	0	0	0	1	1	0	0	0	3	0.790	
	LockBit	36	5	94	2	1	0	1	8	4	0	0	7	0.539	
	REvil	0	1	2	192	0	0	1	1	2	0	0	12	0.919	
	Ryuk	1	0	0	1	157	0	0	32	11	12	0	1	0.758	
	WannaCry	1	0	5	0	0	185	5	0	0	0	0	4	0.907	
Benign	AESCrypt	0	2	12	0	0	19	117	1	0	0	1	3	0.669	0.905
	Firefox	0	1	1	2	9	0	40	126	37	1	1	12	0.568	
	Idle	1	0	0	2	3	0	1	9	101	23	0	3	0.594	
	Office	0	0	3	1	21	3	1	26	39	166	2	0	0.714	
	SDelete	0	1	0	0	0	1	0	2	0	0	198	1	0.978	
	Zip	1	0	8	7	0	0	22	0	1	0	0	129	0.746	

(b) Storage features, χ_s

Predicted class

Actual class		Ransomware						Benign						F score in 12 classes	F score in 2 classes
		Conti	Darkside	LockBit	REvil	Ryuk	WannaCry	AESCrypt	Firefox	Idle	Office	SDelete	Zip		
Ransomware	Conti	73	3	69	0	0	12	6	1	1	2	1	7	0.382	0.861
	Darkside	106	143	77	7	0	0	4	0	0	0	21	0	0.517	
	LockBit	12	3	33	1	9	9	14	5	3	2	5	2	0.209	
	REvil	11	21	20	155	0	0	1	2	0	0	3	1	0.765	
	Ryuk	0	0	0	1	175	1	1	16	3	22	2	4	0.812	
	WannaCry	0	0	0	2	0	111	22	7	0	0	0	26	0.617	
Benign	AESCrypt	0	0	0	2	0	5	80	1	1	0	1	8	0.505	0.856
	Firefox	0	1	3	3	8	0	21	139	3	10	4	2	0.702	
	Idle	1	0	0	19	0	0	1	7	175	22	0	0	0.814	
	Office	0	0	0	1	4	3	2	14	19	129	0	0	0.719	
	SDelete	4	24	16	0	10	1	1	8	0	0	167	1	0.764	
	Zip	0	0	0	0	0	50	66	2	0	0	1	122	0.589	

(c) Memory features, χ_m

Fig. C.2. Confusion matrices and F scores when we trained the ransomware detectors using 100% of the group **(1-2)**, Core i3 12100 CPU, and predicted a class using the randomly selected 40% of the group **(1-1)**, Celeron G3920 CPU.

point of view and the hypervisor's point of view; Dangl et al. presented a high-performance caching mechanism that resolves inconsistency problem in virtual machine introspection (Dangl et al., 2023). Our monitoring function of memory access patterns uses Second Level Address Translation (SLAT) mechanisms (i.e., Intel EPT); our system clears the address translation cache in the Translation Lookaside Buffer (TLB) every 30 s to forcefully cause address translations. Our method cannot collect all memory access patterns; once a Guest Physical Address (GPA) is translated into a Host Physical Address (HPA), the mapping (i.e., the address translation cache) between the GPA and the HPA is stored in TLB. While the mapping exists in TLB, we cannot monitor memory accesses on the same GPA since our monitoring system depends on the SLAT mechanism. By clearing TLB every 30 s, we can collect access patterns on the GPA previously stored in TLB. The experimental result of the performance degradation of our thin hypervisor was 18%; in contrast, that of performance degradation of Microsoft's hypervisor with Virtualization Based Security (VBS) was about 24% (Hirano and Kobayashi, 2022). Although clearing TLB in less than 30 s increases the number of memory access patterns to be collected, it causes more performance degradation. We must carefully examine the balance between performance degradation caused by monitoring systems and monitoring coverage.

General utility. We finally discuss to what extent the dataset's findings can be applied to measures taken by other hypervisors or directly within an operating system. This paper employed a Type-I thin hypervisor named BitVisor (Shinagawa et al., 2009). Our memory access pattern monitor was implemented using hardware-assisted memory virtualization technology named Intel's Extended Page Table (EPT) (Intel Corporation, 2024); therefore, the same system can be implemented in many modern hypervisors, including Kernel-based Virtual Machine (KVM) that employs hardware-based memory virtualization such as Intel's EPT. On the other hand, our memory access pattern monitor cannot be implemented in the operating system securely since the OS's page tables (i.e., First Level Address Translation mechanism in

```
1:  procedure calcurate_storage_feature(T_d, T_window, ε)
2:      t ← 0;
3:      lines_{s,r} ← read_all_lines("ata_read.csv");
4:      lines_{s,w} ← read_all_lines("ata_write.csv");
5:      while t ≤ ( T_d - T_window )
6:          lines_{s,r,t} ← extract_lines(lines_{s,r}, t, T_window);
7:          lines_{s,w,t} ← extract_lines(lines_{s,w}, t, T_window);
8:          T_{s,r} ← calc_throughput(lines_{s,r,t});
9:          T_{s,w} ← calc_throughput(lines_{s,w,t});
10:         V_{s,r} ← calc_LBA_variance(lines_{s,r,t});
11:         V_{s,w} ← calc_LBA_variance(lines_{s,w,t});
12:         H_{s,w} ← calc_Shannon_entropy(lines_{s,w,t});
13:         x_s ← {T_{s,r}, T_{s,w}, V_{s,r}, V_{s,w}, H_{s,w}}
14:         χ_s.append(x_s);
15:         t ← t + ε;
16:     return(χ_s);
```

Fig. D.1. Data preprocessing algorithm to create storage feature χ_s.

the OS layer) are not protected from attackers who obtained the OS's privileged accesses. In contrast, our memory access pattern monitor is protected from attackers who obtained OS's privilege accesses since hypervisor's page tables (i.e., Second Level Address Translation mechanism in hypervisor layer) such as EPTs are isolated from the OS using hardware-based virtualization technology.

On the other hand, our storage access pattern monitor was implemented as the extended function of BitVisor's Advanced Host Controller Interface (AHCI) *parapass-through* driver. The concept of *parapass-through* driver was presented in the original BitVisor paper (Shinagawa et al., 2009); the *parapass-throgh* drivers mediate access to the subjected I/O devices (e.g., Solid State Drive) for enforcing security, while most of the I/O accesses from the guest OS to pass-through the hypervisor to minimize the monitoring performance overhead. Although BitVisor's minimum design reduces the monitoring overhead, our storage access pattern monitor can be implemented as an extended function of device drivers of other Type-I hypervisors, such as KVM and VMware ESXi. In contrast, our storage access pattern monitor cannot be implemented securely in the OS layer since the OS's device drivers can be compromised by attackers who obtain the OS's privileged accesses. Thus, both memory and storage access pattern monitors can be implemented in the OS layer by intercepting page faults and by extending the function of the OS's device driver; however, they cannot provide advanced defense-in-depth defenses like our presented system.

8. Future work

The future work includes (1) expanding the dataset by adding storage and memory access patterns of the ransomware samples belonging to different families in order to improve the applicability of the proposed detection method, (2) expanding the dataset by adding storage and memory access patterns on the machines of various hardware configurations and OSs to improve the applicability of the proposed detection method, (3) increasing the detection rate while reducing the detection time by improving our preprocessing method and deep-learning model to counter the latest high-speed ransomware encryption function, (4) improving the detection performance when ransomware is executed simultaneously with encryption, secure delete, compression programs by considering context switching between processes, and finally (5) improving computational efficiency to achieve real-time detection. In particular, implementing the deep-learning model's prediction and updating function directly into the hypervisor with low computational overhead is a challenging open issue.

Another crucial future work is detecting double extortion ransomware attacks that exfiltrate confidential data without or before encrypting victim data. Although encrypting victim files is executed by ransomware samples automatically (i.e., ransomware programs include the functions that list directories and files and encrypt them), exfiltrating victim files is executed by attackers manually. For example, Conti ransomware playbook (Cisco Talos Blog, 2021), which was leaked from the Conti ransomware group, is a guide for attackers using Conti ransomware; it describes step-by-step procedures from company revenue search to confidential data upload to cloud storage, using open source program `rclone` with low bandwidth option, without getting alerts from Data Loss Prevention (DLP) systems in the target company. We have developed our hypervisor's novel network monitoring function and are currently evaluating how to detect double extortion ransomware. In our following paper, we will release the new dataset, which includes not only storage and memory behavioral features but also low-level network features obtained using the hypervisor.

9. Conclusion

This paper presented a new open dataset named RanSMAP, which contains low-level storage and memory access patterns collected using a thin hypervisor; in contrast, our previous RanSAP dataset contained only low-level storage access patterns (Hirano et al., 2022). We examined our deep-learning-based ransomware detector using the RanSMAP dataset. The experimental results showed that the ransomware detectors using storage and memory access patterns increased the F score by 2.3% compared to those using only storage access patterns. The improvement due to introducing memory access patterns might seem small; however, our presented detector improved the ransomware detection rates when REvil ransomware is executed with Firefox and Office simultaneously (e.g., 42% increase for REvil and Firefox, 18% increase for REvil and Office), as shown in Fig. 15. In addition, as shown in Fig. 14, when the detector was trained using both storage and memory features χ, the average recall in 12 classes was 11.4% improved than only storage features χ_s. However, the detection performance improvements often depend on samples; we, therefore, need further experiments to discuss the contribution of memory and storage features.

On the other hand, we confirmed that the ransomware detector trained using access patterns on the newer CPU, Core i3 12100, could detect those on the older CPU, Celeron G3920; however, the opposite was more difficult. Changing the generation of DDR RAM modules and the motherboard decreased the F scores of ransomware detectors using storage access patterns more than memory access patterns. The number of CPU cores significantly impacted the F scores of ransomware detectors using memory access patterns more than those using storage access patterns. In the experiments detecting ransomware variants, the F scores using memory access patterns were better than others; the results indicated that the ensemble approach could be used. We did not observe the temporal evolutionary trends in detecting ransomware variants. The simultaneous execution of REvil ransomware with Office and a Firefox web browser had better detection performance than REvil ransomware with AESCrypt, SDelete, and Zip programs; these results indicated that ransomware that mimics the behaviors of legitimate encryption, secure delete, and compression programs is more challenging to detect. Our RanSMAP dataset and hypervisor-based monitoring system with deep-learning-based ransomware detector presented in this paper can be a building block of defense-in-depth defense systems against today's sophisticated ransomware attacks. They provide additional protection in the hypervisor layer to the OS-level ransomware detection mechanisms.

```
procedure calcurate_memory_feature(T_d, T_window, ε)
    t ← 0;
    lines_{m,r} ← read_all_lines("mem_read.csv");
    lines_{m,w} ← read_all_lines("mem_write.csv");
    lines_{m,rw} ← read_all_lines("mem_readwrite.csv");
    lines_{m,x} ← read_all_lines("mem_exec.csv");
    while t ≤ ( T_d - T_window )
        lines_{m,r,t} ← extract_lines(lines_{m,r}, t, T_window);
        lines_{m,w,t} ← extract_lines(lines_{m,w}, t, T_window);
        lines_{m,rw,t} ← extract_lines(lines_{m,rw}, t, T_window);
        lines_{m,x,t} ← extract_lines(lines_{m,x}, t, T_window);
        H_{m,w}, H_{m,rw} ← calc_Shannon_entropy(lines_{m,w,t}, lines_{m,rw,t});
        C_{4KB,r}, C_{4KB,w}, C_{4KB,rw}, C_{4KB,x} ← count_4KB_page_access(lines_{m,r,t}, lines_{m,w,t}, lines_{m,rw,t}, lines_{m,x,t});
        C_{2MB,r}, C_{2MB,w}, C_{2MB,rw}, C_{2MB,x} ← count_2MB_page_access(lines_{m,r,t}, lines_{m,w,t}, lines_{m,rw,t}, lines_{m,x,t});
        C_{MMIO,r}, C_{MMIO,w}, C_{MMIO,rw}, C_{MMIO,x} ← count_MMIO_page_access(lines_{m,r,t}, lines_{m,w,t}, lines_{m,rw,t}, lines_{m,x,t});
        V_{m,r}, V_{m,w}, V_{m,rw}, V_{m,x} ← calc_GPA_variance(lines_{m,r,t}, lines_{m,w,t}, lines_{m,rw,t}, lines_{m,x,t});
        x_m ← {H_{m,w}, H_{m,rw}, C_{4KB,r}, C_{4KB,w}, C_{4KB,rw}, C_{4KB,x}, C_{2MB,r}, C_{2MB,w}, C_{2MB,rw}, C_{2MB,x}, C_{MMIO,r}, C_{MMIO,w}, C_{MMIO,rw},
                C_{MMIO,x}, V_{m,r}, V_{m,w}, V_{m,rw}, V_{m,x}}
        χ_m.append(x_m);
        t ← t + ε;
    return(χ_m);
```

Fig. D.2. Data preprocessing algorithm to create memory feature χ_m.

CRediT authorship contribution statement

Manabu Hirano: Writing – review & editing, Writing – original draft, Visualization, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Ryotaro Kobayashi:** Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Manabu Hirano reports financial support was provided by Japan Society for the Promotion of Science.

Acknowledgments

The authors gratefully acknowledge constructive comments by the anonymous reviewers. The authors thank Noriaki Kawane, Kyo Ban, Syuya Kawai, Yuta Fukino, Taishi Naka, Yuki Hiraoka, Kensyo Ishibashi, Eitaro Osugi, Reishi Kondo, Kaito Motokawa, and Tomohito Yamaguchi for their support in constructing the dataset. The authors gratefully thank the developers of BitVisor. This work was supported by JSPS KAKENHI Grant Number JP23K11114.

Appendix A. Acronyms

Table A.1 shows the acronyms used in this paper.

Table A.1
The acronyms used in this paper.

AHCI	Advanced Host Controller Interface
API	Application Programming Interface
ATA	Advanced Technology Attachment
C&C	Command and Control
CSV	Comma Separated Vales
DDR	Double Data Rate
DMA	Direct Memory Access
DNS	Domain Name System
EPT	Extended Page Table
GPA	Guest Physical Address
GVA	Guest Virtual Address
HDD	Hard Disk Drive
HPA	Host Physical Address
I/O	Input and Output
IRP	I/O Request Packets
JSON	JavaScript Object Notation
KVM	Kernel-based Virtual Machine
LAN	Local Area Network
LBA	Logical Block Address
LSTM	Long Short-Term Memory
ML	Machine Learning
MMIO	Memory Mapped Input and Output
NVMe	Non-Volatile Memory Express
OS	Operating System
PE	Portable Executable
RAM	Random Access Memory
RDP	Remote Desktop Protocol
SLAT	Second-Level Address Translation
SMB	Server Message Block
SSD	Solid State Drive
TLB	Translation Lookaside Buffer
UDP	User Datagram Protocol
UEFI	Unified Extensible Firmware Interface
VBS	Virtualization Based Security
VM	Virtual Machine
VMX	Virtual Machine eXtensions
VPN	Virtual Private Network

Appendix B. Confusion matrices of baseline ransomware detector

Fig. B.1 shows confusion matrices and F scores of the baseline ransomware detectors; we trained them using randomly selected 80% of the group **(1)** and predicted a class using the remaining 20% of the group **(1)** (i.e., $D_{test} = \{\mathbf{(1)}\}$ and $D_{train} = \{\mathbf{(1)}\}$). Please note that the F scores are not identical to those in Fig. 7 and those in Fig. 8 since we created ransomware detectors and predicted a class another ten times to create these confusion matrices.

Appendix C. Confusion matrices of ransomware detector by CPU

Fig. C.1 shows confusion matrices and F scores when we trained the ransomware detectors using 100% of the group **(1-1)**, the access patterns on Celeron G3920 CPU, and predicted a class using randomly selected 40% of the group **(1-2)**, Core i3 12100 CPU. The directory

groups are shown in Table 8 (i.e., D_{test} = {**(1-2)**} and D_{train} = {**(1-1)**}). Fig. C.2 shows confusion matrices and F scores when we trained the ransomware detectors using 100% of the group **(1-2)**, the access patterns on Core i3 12100 CPU, and predicted a class using randomly selected 40% of the group **(1-1)**, Celeron G3920 CPU (i.e., D_{test} = {**(1-1)**} and D_{train} = {**(1-2)**}). Please note that the F scores in Figs. C.1 and C.2 are not identical to those in Fig. 10 since we created ransomware detectors and predicted a class another ten times to create these confusion matrices.

Appendix D. Data preprocessing algorithms

Fig. D.1 and Fig. D.2 show the detailed data preprocessing algorithms.

Appendix E. Supplementary data

The RanSMAP dataset can be found online at the following links:

- RanSMAP dataset (GitHub)
- RanSMAP dataset (Kaggle)

Data availability

I have shared the link to my dataset above.

References

Alhawi, O.M., Baldwin, J., Dehghantanha, A., 2018. Leveraging machine learning techniques for windows ransomware network traffic detection. Cyber Threat Intell.: Adv. Inf. Secur. 70, 93–106. http://dx.doi.org/10.1007/978-3-319-73951-9_5.

Allix, K., Bissyandé, T.F., Klein, J., Traon, Y.Le., 2016. AndroZoo: Collecting millions of android apps for the research community. In: Proceedings of the 13th International Conference on Mining Software Repositories. ACM, Austin Texas, pp. 468–471. http://dx.doi.org/10.1145/2901739.2903508.

Alzahrani, S., Xiao, Y., Sun, W., 2022. An analysis of conti ransomware leaked source codes. IEEE Access 10, 100178–100193. http://dx.doi.org/10.1109/ACCESS.2022.3207757.

Amer, E., El-Sappagh, S., 2022. Robust deep learning early alarm prediction model based on the behavioural smell for android malware. Comput. Secur. 116, 102670. http://dx.doi.org/10.1016/j.cose.2022.102670.

Amer, E., Zelinka, I., El-Sappagh, S., 2021. A multi-perspective malware detection approach through behavioral fusion of API call sequence. Comput. Secur. 110, 102449. http://dx.doi.org/10.1016/j.cose.2021.102449.

ANY.RUN, 2017. Interactive malware hunting service. https://any.run/. (Accessed 19 2024).

Barr-Smith, F., Ugarte-Pedrero, X., Graziano, M., Spolaor, R., Martinovic, I., 2021. Survivalism: Systematic analysis of windows malware living-off-the-land. In: 2021 IEEE Symposium on Security and Privacy (SP). IEEE, San Francisco, CA, USA, pp. 1557–1574. http://dx.doi.org/10.1109/SP40001.2021.00047.

Beaman, C., Barkworth, A., Akande, T.D., Hakak, S., Khan, M.K., 2021. Ransomware: recent advances, analysis, challenges and future research directions. Comput. Secur. 111, 102490. http://dx.doi.org/10.1016/j.cose.2021.102490.

Berardi, D., Giallorenzo, S., Melis, A., Melloni, S., Onori, L., Prandini, M., 2023. Data flooding against ransomware: concepts and implementations. Comput. Secur. 131, 103295. http://dx.doi.org/10.1016/j.cose.2023.103295.

Berrueta, E., Morato, D., Magaña, M., 2020. Open repository for the evaluation of ransomware detection tools. IEEE Access 8, 65658–65669. http://dx.doi.org/10.1109/ACCESS.2020.2984187.

Botacin, M., Ceschin, F., Sun, R., Oliveira, D., Grégio, A., 2021. Challenges and pitfalls in malware research. Comput. Secur. 106, 102287. http://dx.doi.org/10.1016/j.cose.2021.102287.

Cabaj, K., Gregorczyk, M., Mazurczyk, W., 2018. Software-defined networking-based crypto ransomware detection using HTTP traffic characteristics. Comput. Electr. Eng. 66, 353–368. http://dx.doi.org/10.1016/j.compeleceng.2017.10.012.

Cen, M., Jiang, F., Qin, X., Jiang, Q., Doss, R., 2024. Ransomware early detection: A survey. Comput. Netw. 239, 110138. http://dx.doi.org/10.1016/j.comnet.2023.110138.

Chen, Z.G., Kang, H.S., Yin, S.N., Kim, S.R., 2017. Automatic ransomware detection and analysis based on dynamic API calls flow graph. In: Proceedings of the International Conference on Research in Adaptive and Convergent Systems. pp. 196–201. http://dx.doi.org/10.1145/3129676.3129704.

Chronicle, 2004. Virustotal. https://www.virustotal.com/. (Accessed 19 2024).

Cisco Talos Blog, 2021. Translated: Talos' insights from the recently leaked conti ransomware playbook. https://blog.talosintelligence.com/conti-leak-translation/. (Accessed 05 2024).

Continella, A., Guagnelli, A., Zingaro, G., Pasquale, G.D., Barenghi, A., Zanero, S., Maggi, F., 2016. Shieldfs: A self-healing, ransomware-aware filesystem. In: Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC 2016). pp. 336–347. http://dx.doi.org/10.1145/2991079.2991110.

Corvus Forensics, 2007. Virusshare. https://virusshare.com/. (Accessed 19 2024).

Cusack, G., Michel, O., Keller, E., 2018. Machine learning-based detection of ransomware using SDN. In: Proceedings of the 2018 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization. pp. 1–6. http://dx.doi.org/10.1145/3180465.3180467.

Dangl, T., Sentanoe, S., Reiser, H.P., 2023. Vmifresh: efficient and fresh caches for virtual machine introspection. Comput. Secur. 135, 103527. http://dx.doi.org/10.1016/j.cose.2023.103527.

Diamantopolous, D., Pletka, R., Sarafijanovic, S., Reddy, A.L.N., Pozidis, H., 2024. WannaLaugh: a configurable ransomware emulator – learning to mimic malicious storage traces. arXiv:2403.07540.

Fernando, D.W., Komninos, N., 2024. Fesad ransomware detection framework with machine learning using adaption to concept drift. Comput. Secur. 137, 103629. http://dx.doi.org/10.1016/j.cose.2023.103629.

Filiz, B., Arief, B., Cetin, O., Hernandez-Castro, J., 2021. On the effectiveness of ransomware decryption tools. Comput. Secur. 111, 102469. http://dx.doi.org/10.1016/j.cose.2021.102469.

Gama, J., Žliobaite, I., Bifet, A., Pechenizkiy, M., Bouchachia, A., 2014. A survey on concept drift adaptation. ACM Comput. Surv. (CSUR) 46, 1–37. http://dx.doi.org/10.1145/2523813.

Garfinkel, S.L., 2009. GovDocs1 - digital corpora. https://digitalcorpora.org/corpora/file-corpora/files/. (Accessed 19 2024).

Garfinkel, S., Farrell, P., Roussev, V., Dinolt, G., 2009. Bringing science to digital forensics with standardized forensic corpora. Digit. Invest. 6, S2–S11. http://dx.doi.org/10.1016/j.diin.2009.06.016.

Gulmez, S., Kakisim, A.Gorgulu., Sogukpinar, I., 2024. Xran: explainable deep learning-based ransomware detection using dynamic analysis. Comput. Secur. 139, 103703. http://dx.doi.org/10.1016/j.cose.2024.103703.

Hampton, N., Baig, Z., Zeadally, S., 2018. Ransomware behavioural analysis on windows platforms. J. Inf. Secur. Appl. 40, 44–51. http://dx.doi.org/10.1016/j.jisa.2018.02.008.

Harang, R., Rudd, E.M., 2020. SOREL-20m: a large scale benchmark dataset for malicious PE detection. http://dx.doi.org/10.48550/arXiv.2012.07634.

Herrera-Silva, J.A., Hernández-Álvarez, M., 2023. Dynamic feature dataset for ransomware detection using machine learning algorithms. Sensors 23 (1053), http://dx.doi.org/10.3390/s23031053.

Hirano, M., 2022. Open repository of ransap dataset. https://github.com/manabu-hirano/RanSAP. (Accessed 19 2024).

Hirano, M., 2024a. Open repository of ransmap dataset. https://github.com/manabu-hirano/RanSMAP.

Hirano, M., 2024b. Ransmap (2024) ransomware behavioral features. https://www.kaggle.com/datasets/hiranomanabu/ransmap-2024-ransomware-behavioral-features.

Hirano, M., Hodota, R., Kobayashi, R., 2022. Ransap: an open dataset of ransomware storage access patterns for training machine learning models. Forens. Sci. Int.: Digit. Invest. 40, 301314. http://dx.doi.org/10.1016/j.fsidi.2021.301314.

Hirano, M., Kobayashi, R., 2019. Machine learning based ransomware detection using storage access patterns obtained from live-forensic hypervisor. In: 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS 2019). pp. 1–6. http://dx.doi.org/10.1109/IOTSMS48152.2019.8939214.

Hirano, M., Kobayashi, R., 2022. Machine learning-based ransomware detection using low-level memory access patterns obtained from live-forensic hypervisor. In: 2022 IEEE International Conference on Cyber Security and Resilience (CSR). pp. 323–330. http://dx.doi.org/10.1109/CSR54599.2022.9850340.

Hull, G., John, H., Arief, B., 2019. Ransomware deployment methods and analysis: Views from a predictive model and human responses. Crime Sci. 8, 1–22. http://dx.doi.org/10.1186/s40163-019-0097-9.

Intel Corporation, 2024. Intel® 64 and IA-32 architectures software developer's manual. In: Volume 3C: System Programming Guide Part 3.

Jung, S., Won, Y., 2018. Ransomware detection method based on context-aware entropy analysis. Soft Comput. 22, 6731–6740. http://dx.doi.org/10.1007/s00500-018-3257-z.

Kharaz, A., Arshad, S., Mulliner, C., Robertson, W., Kirda, E., 2016. UNVEIL: A large-scale, automated approach to detecting ransomware. In: 25th USENIX Security Symposium 2016. pp. 757–772. http://dx.doi.org/10.5555/3241094.3241153.

Kharraz, A., Kirda, E., 2017. Redemption: real-time protection against ransomware at end-hosts. In: International Symposium on Research in Attacks, Intrusions, and Defenses (RAID 2017). Springer International Publishing, pp. 98–119. http://dx.doi.org/10.1007/978-3-319-66332-6_5.

Kolodenker, E., Koch, W., Stringhini, G., Egele, M., 2017. Paybreak: defense against cryptographic ransomware. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security. pp. 599–611. http://dx.doi.org/10.1145/3052973.3053035.

Kumar, S., et al., 2020. An emerging threat fileless malware: A survey and research challenges. Cybersecurity 3, 1–12. http://dx.doi.org/10.1186/s42400-019-0043-x.

Ma, B., Yang, Y., Li, J., Zhang, F., Shen, W., Zhou, Y., Ma, J., 2023. Travelling the hypervisor and SSD: a tag-based approach against crypto ransomware with fine-grained data recovery. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. ACM, Copenhagen Denmark, pp. 341–355. http://dx.doi.org/10.1145/3576915.3616665.

van der Maaten, L., Hinton, G., 2008. Visualizing data using t-SNE. J. Mach. Learn. Res. 9, 2579–2605.

Mahdavifar, S., Abdul Kadir, A.F., Fatemi, R., Alhadidi, D., Ghorbani, A.A., 2020. Dynamic android malware category classification using semi-supervised deep learning.

McIntosh, T., Kayes, A.S.M., Chen, Y.P.P., Ng, A., Watters, P., 2021. Ransomware mitigation in the modern era: A comprehensive review, research challenges, and future directions. ACM Comput. Surv. 54, 1–36. http://dx.doi.org/10.1145/3479393.

McIntosh, T., Kayes, A.S.M., Chen, Y.P.P., Ng, A., Watters, P., 2023. Applying staged event-driven access control to combat ransomware. Comput. Secur. 128, 103160. http://dx.doi.org/10.1016/j.cose.2023.103160.

Microsoft, 2021. About file system filter drivers - windows drivers. https://learn.microsoft.com/en-us/windows-hardware/drivers/ifs/about-file-system-filter-drivers. (Accessed 19 2024).

More, A., Tapaswi, S., 2014. Virtual machine introspection: Towards bridging the semantic gap. J. Cloud Comput. 3, 1–14. http://dx.doi.org/10.1186/s13677-014-0016-2.

Moussaileb, R., Cuppens, N., Lanet, J.L., Bouder, H.L., 2021. A survey on windows-based ransomware taxonomy and detection mechanisms. ACM Comput. Surv. 54, 1–36. http://dx.doi.org/10.1145/3453153.

Okamura, K., Oyama, Y., 2012. Controlling the speed of virtual time for malware deactivation. In: Proceedings of the Asia-Pacific Workshop on Systems. ACM, Seoul Republic of Korea, pp. 1–6. http://dx.doi.org/10.1145/2349896.2349902.

Oz, H., Aris, A., Levi, A., Uluagac, A.S., 2022. A survey on ransomware: evolution, taxonomy, and defense solutions. ACM Comput. Surv. 54, 1–37. http://dx.doi.org/10.1145/3514229.

Packetizer Inc., 2007. AES crypt - advanced file encryption. https://www.aescrypt.com/. (Accessed 19 2024).

Palo Alto Networks, 2022. BlueSky ransomware: fast encryption via multithreading. https://unit42.paloaltonetworks.com/bluesky-ransomware/. (Accessed 19 2024).

Roy, K.C., Chen, Q., 2021. DeepRan: attention-based bilstm and crf for ransomware early detection and classification. Inf. Syst. Front. 23, 299–315. http://dx.doi.org/10.1007/s10796-020-10017-4.

Russinovich, M.E., Margosis, A., 2011. Windows sysinternals administrator's reference. Microsoft Press.

Scaife, N., Carter, H., Traynor, P., Butler, K.R.B., 2016. Cryptolock (and drop it): Stopping ransomware attacks on user data. In: 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS 2016). pp. 303–312. http://dx.doi.org/10.1109/ICDCS.2016.46.

Schumilo, S., Aschermann, C., Abbasi, A., Wörner, S., Holz, T., 2021. Nyx: greybox hypervisor fuzzing using fast snapshots and affine types. In: 30th USENIX Security Symposium 2021. pp. 2597–2614.

Shannon, C.E., 1948. A mathematical theory of communication. Bell Syst. Tech. J. 27, 379–423. http://dx.doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Shaukat, S.K., Ribeiro, V.J., 2018. RansomWall: A layered defense system against cryptographic ransomware attacks using machine learning. In: 2018 10th International Conference on Communication Systems & Networks (COMSNETS). pp. 356–363. http://dx.doi.org/10.1109/COMSNETS.2018.8328219.

Shinagawa, T., Eiraku, H., Tanimoto, K., Omote, K., Hasegawa, S., Horie, T., Hirano, M., Kourai, K., Oyama, Y., Kawai, E., et al., 2009. BitVisor: a thin hypervisor for enforcing i/O Device Security. In: Proceedings of the 2009 ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE 2009). pp. 121–130. http://dx.doi.org/10.1145/1508293.1508311.

Singh, N., Tripathy, S., 2024. It's too late if exfiltrate: early stage android ransomware detection. Comput. Secur. 141, 103819. http://dx.doi.org/10.1016/j.cose.2024.103819.

Tang, F., Ma, B., Li, J., Zhang, F., Su, J., Ma, J., 2020. RansomSpector: an introspection-based approach to detect crypto ransomware. Comput. Secur. 97, 101997. http://dx.doi.org/10.1016/j.cose.2020.101997.

Widmer, G., Kubat, M., 1996. Learning in the presence of concept drift and hidden contexts. Mach. Learn. 23, 69–101. http://dx.doi.org/10.1007/BF00116900.

Witte, T.N., 2020. Phantom malware: conceal malicious actions from malware detection techniques by imitating user activity. IEEE Access 8, 164428–164452. http://dx.doi.org/10.1109/ACCESS.2020.3021743.

Yuste, J., Pastrana, S., 2021. Avaddon ransomware: an in-depth analysis and decryption of infected systems. Comput. Secur. 109, 102388. http://dx.doi.org/10.1016/j.cose.2021.102388.

Zhou, C., Guo, L., Hou, Y., Ma, Z., Zhang, Q., Wang, M., Liu, Z., Jiang, Y., 2023. Limits of i/o based ransomware detection: an imitation based attack. In: 2023 IEEE Symposium on Security and Privacy (SP). IEEE, San Francisco, CA, USA, pp. 2584–2601. http://dx.doi.org/10.1109/SP46215.2023.10179372.

Manabu Hirano was born in Hokkaido, Japan in 1976. He received his B.E. degree in information engineering from the Toyohashi University of Technology, Toyohashi, Japan, in 2000. He received his M.E. and D.E. degrees in information engineering from the Nara Institute of Science and Technology (NAIST), Nara, Japan, in 2002 and 2008. He had worked for TOSHIBA's Internet-business venture department since 2002. Since 2004, He has worked for the National Institute of Technology, Toyota College, as an Assistant Professor in the Department of Information and Computer Engineering. From 2009 to 2011, he was a Senior Lecturer. From 2011 to 2022, he was an Associate Professor. In addition, he studied at the University of Kent, Canterbury, UK, as a Visiting Senior Lecturer in the School of Computing between June 2012 and January 2013. He has been a Professor at the National Institute of Technology, Toyota College, in the Department of Information and Computer Engineering since 2023. His research interests include cyber security, machine learning, and hypervisor-based defense mechanisms.

Ryotaro Kobayashi received his B.E., M.E., and D.E. degrees from Nagoya University in 1995, 1997, and 2001, respectively. He had been a research assistant at Nagoya University from 2000 to 2008, a lecturer at Toyohashi University from 2008 to 2015, an associate professor at Toyohashi University from 2016 to 2017, and an associate professor at Kogakuin University from 2017 to 2019. He is currently a professor at Kogakuin University. His research interests include cyber security, machine learning, and hardware-based defense mechanisms.